STUDY GUIDE

NLP Zero to Hero

Hindi Text Restoration and Translation

From first principles to a working three-stage pipeline: punctuation, sentence ordering, and translation. Every model, every source file, every notebook cell, and every measured result, explained.

Pipeline: mBERT + LoRA → pairwise ordering → IndicTrans2

Constraints: no LLM · 1.5 GB GPU memory · three models, loaded one at a time

Repository: github.com/Vaibhavvv2524/nlp-zero-to-hero

Verified locally: macOS M4 (MPS), subset adapters, executed demo notebook

This guide explains the project from first principles: what NLP is, how text becomes tokens and vectors, how transformers and LoRA work, how each of the three assignment stages is built, what every source file does, what the notebooks do cell by cell, what the measured results mean, and how to defend the work in a viva.

It is written for someone who is starting from zero but wants to finish with a genuinely deep understanding. Nothing important is hidden behind “it just works”.

> **Scope of the claims in this guide.** All numbers come from the JSON reports and training summaries in this repository. The punctuation and ordering adapters that produced the reported pipeline outputs were trained on 4,000 punctuation windows (2 epochs) and 4,000 ordering pairs (1 epoch) on Apple MPS. The full-data Colab recipe (25,000 windows, 40,000 pairs) is documented but was not completed locally. Wherever a number depends on that distinction, the guide says so explicitly.

Contents

Part 0 — Orientation

1. The assignment in plain English
2. The seven inputs and the four outputs
3. The big picture: data flow and model flow
4. Vocabulary for the rest of the guide

Part 0.5 — The approach we chose (read this first)

A.1 Why decompose the problem at all?

A.2 The eight principles behind the chosen approach

A.3 The approach at a glance

A.4 What we explicitly rejected

A.5 How the approach satisfies the assignment constraints

A.6 Why this approach is defensible in a viva

Part I — NLP foundations from zero

5. What is natural language processing?

5.1 Rule-based, statistical, neural, and LLM approaches

5.2 Where this project sits

6. From text to tokens

6.1 Text is Unicode, not bytes

6.2 Word tokenization

6.3 Sentence tokenization

6.4 Subword tokenization

7. Embeddings and vector representations

7.1 One-hot and bag-of-words

7.2 TF-IDF

7.3 Static word embeddings

7.4 Contextual embeddings

7.5 Cosine similarity

8. Transformers and BERT

8.1 The basic neural network

8.2 Attention

8.3 Encoder, decoder, and sequence-to-sequence

8.4 BERT and mBERT

8.5 Task heads

8.6 Why mBERT for this project

9. Transfer learning, fine-tuning, and LoRA

9.1 Pretraining and fine-tuning

9.2 Parameter-efficient fine-tuning

9.3 The LoRA idea

9.4 LoRA hyperparameters

9.5 Adapter merging

10. Training mechanics

10.1 Data splits

10.2 Batch, step, epoch

10.3 Learning rate, warmup, weight decay

10.4 Gradient accumulation and checkpointing

10.5 Mixed precision

10.6 Seeds and reproducibility

10.7 Class weights

10.8 Model selection

10.9 Exact training settings

10.10 Diagnosing training

11. Inference and decoding

11.1 Argmax and softmax

11.2 Sliding windows and triangular weighting

11.3 Batching

11.4 Beam search

11.5 Ordering decoders

11.6 Complexity

12. Metrics: what the numbers mean

12.1 Confusion matrix, accuracy, precision, recall, F1

12.2 Macro, micro, weighted

12.3 Boundary F1

12.4 Per-class F1 and the accuracy trap

12.5 Pair accuracy and macro F1 for ordering

12.6 Kendall's tau

12.7 BLEU

12.8 chrF

12.9 Translation results

13. Data ethics, licensing, and responsible claims

13.1 Data source

13.2 Model licenses and gating

13.3 Privacy

13.4 Bias and failure modes

13.5 Honest reporting

14. The end-to-end mental model

Part II — The project in depth

15. Repository tour

15.1 What each source file does

15.2 The baseline/ folder

15.3 The lessons/ folder

16. Environment and dependency saga

16.1 Why each pin exists

16.2 The MPS attention problem

16.3 The gated IndicTrans2 problem

16.4 The data-preparation thread hang

16.5 Two local environments

16.6 Setup commands

17. Data preparation deep dive

17.1 Streaming Wikipedia

17.2 Parsing an article

17.3 Punctuation windows

17.4 Splitting by article

17.5 Capping examples

17.6 Ordering pairs

17.7 The held-out ordering articles

17.8 Output schemas

17.9 Reproducing data preparation

18. Stage 1: Punctuation restoration

18.1 Problem formulation

18.2 Why token classification

18.3 Subword alignment

18.4 Model and head

18.5 Class-weighted loss

18.6 Inference: sliding windows and triangular weights

18.7 Rebuilding sentences

18.8 What the model learns

18.9 Worked example from the demo notebook

18.10 Known limitations

19. Stage 2: Sentence ordering

19.1 Problem formulation

19.2 Why pairwise

19.3 Building balanced pairs

19.4 Model

19.5 Inference: the score matrix

19.6 Decoder 1: Borda

19.7 Decoder 2: clusters

19.8 Decoder 3: beam

19.9 TF-IDF ablation

19.10 Kendall's tau evaluation

19.11 Worked example from the demo notebook

19.12 Known limitations

20. Stage 3: Hindi-to-English translation

20.1 Model

20.2 IndicProcessor

20.3 Generation

20.4 Batching and memory

20.5 Translation quality

20.6 Why not a larger model or an LLM

20.7 Known limitations

21. Pipeline orchestration

21.1 Reading tokens

21.2 Stage 1

21.3 Stage 2

21.4 Stage 3

21.5 Saving

21.6 Failure behavior

21.7 CLI options

22. Evaluation

22.1 Punctuation evaluation

22.2 Ordering evaluation

22.3 Translation evaluation

22.4 Report structure

22.5 Running subsets

22.6 Interpreting the results honestly

23. Reproducibility: from zero to outputs

23.1 Full Colab recipe

23.2 Local verified recipe

23.3 Chronology of the build

23.4 How to verify the results

23.5 What is not done

Part III — The notebooks, cell by cell
24. colab_end_to_end.ipynb — the full recipe
24.1 Cell map
24.2 Cell-by-cell notes
24.3 How to run it
25. pipeline_demo.ipynb — the executed local demo
25.1 Cell map and observed outputs
25.2 Why the demo matters
25.3 How to run it
26. The lesson notebooks and legacy baseline
26.1 Lessons
26.2 Baseline
Part IV — Results, defenses, and reference
27. Results and limitations
27.1 Headline table
27.2 Interpretation
27.3 Limitations, stated plainly
27.4 What would improve it most
28. Design decisions, expanded
D1. Language: Hindi
D2. Tokenization: whitespace, not regex
D3. Punctuation: token classification, not seq2seq
D4. Model: mBERT + LoRA
D5. Label alignment: first subword
D6. Ordering: pairwise binary classification + Borda
D7. Ordering training data: balanced both directions
D8. Translation: IndicTrans2 200M
D9. Memory: staged load/free vs simultaneous
D10. Evaluation: per-stage, honest subsets
D11. Notebooks: recipe + executed demo
D12. No LLM
29. Viva and interview question bank
29.1 Problem and data
29.2 Punctuation
29.3 Ordering
29.4 Translation
29.5 Engineering
Appendices
Appendix A. Command reference
Appendix B. Hyperparameters
Appendix C. Dataset statistics
Appendix D. Report schemas
Appendix E. Dependency error table
Appendix F. References and further study
Appendix G. Study plan

Part 0 — Orientation

1. The assignment in plain English

You are given Hindi text with three problems:

1. **No punctuation and no sentence boundaries.** The words arrive in a flat stream. You must decide where commas, full stops (the Devanagari danda `।`), question marks, and exclamation marks belong.
2. **The sentences are shuffled.** Even after you recover the boundaries, the sentences are not in their original order. You must recover a sensible order.
3. **The text must be translated into English**, and the translation must preserve the grouping.

The required deliverables are, for each input file: the Hindi sentences in recovered order, the same sentences grouped into blocks of five, the English translation of each sentence, and the English groups.

A hard constraint is attached: **no LLM may be used**, and the whole pipeline must fit in a **1.5 GB GPU memory budget**. That is why the project uses two small, task-specific model families rather than an instruction-following chat model.

The final architecture is:

Stage	Question answered	Model	Output
1. Punctuation	Does this word end a sentence, take a comma, or take no mark?	`bert-base-multilingual-cased` + LoRA token classifier	one label per word
2. Ordering	Given sentences A and B, does A come before B?	`bert-base-multilingual-cased` + LoRA pair classifier	probability per pair, decoded into a full order
3. Translation	What is the English for this Hindi sentence?	`ai4bharat/indictrans2-indic-en-dist-200M`	English sentence

> **Key idea.** The project is not one model. It is three models plus a data pipeline, glued together by a small amount of careful engineering. Most of the difficulty is in the data representation, the label alignment, and the decoding logic, not in the model code.

2. The seven inputs and the four outputs

The raw inputs live in `data/raw/sample-1.txt` through `sample-7.txt`. Each file is space-separated tokens. The assignment's tokenizer has replaced some words with the literal token `NULL`; those must be removed before modelling. The files contain no punctuation and no sentence order information.

After `python -m src.pipeline`, each input produces four files:

Output file	What it contains
`outputs/sample-N.sentences_hi.txt`	one punctuated Hindi sentence per line, in recovered order
`outputs/sample-N.groups_hi.txt`	one line per group of five Hindi sentences

Output file	What it contains
`outputs/sample-N.sentences_en.txt`	one English translation per Hindi sentence, same order
`outputs/sample-N.groups_en.txt`	English translations joined into the same groups of five

The produced counts are:

File	Words after removing `NULL`	`NULL` removed	Recovered sentences	Groups of five
sample-1	747	5	52	11
sample-2	768	5	55	11
sample-3	742	6	49	10
sample-4	1,118	4	53	11
sample-5	1,026	9	55	11
sample-6	507	11	80	16
sample-7	846	27	99	20

The last group in a file is allowed to contain fewer than five sentences. Grouping is purely positional: sentences 1-5 become group 1, sentences 6-10 become group 2, and so on.

> **Why sample-6 and sample-7 have so many sentences.** Those files contain dialogue-like text with many short utterances. The punctuation model learned sentence endings from Wikipedia, where complete sentences are usually at least five words long. Short utterances therefore look like sentence endings to the model, and the file is over-segmented. This is a real limitation, not a bug in the grouping code.

3. The big picture: data flow and model flow

```
data/raw/sample-N.txt
        |
        |  read_tokens(): str.split(), drop NULL
        v
flat list of Hindi words
        |
        |  Stage 1: PunctuationRestorer
        |  - 64-word sliding windows, stride 40
        |  - mBERT + LoRA token classifier
        |  - triangular weighted averaging
        v
one label per word  +  punctuated sentences  +  plain sentences
        |
        |  Stage 2: OrderScorer
        |  - all sentence pairs [CLS] A [SEP] B [SEP]
        |  - P(A before B) for every pair
        |  - Borda / clusters / beam decoding
        v
recovered sentence order (a permutation of the sentence indices)
        |
        |  Stage 3: IndicTrans2Translator
        |  - IndicProcessor tags + preprocessing
        |  - 200M encoder-decoder, beam search
        v
English sentences in the same order
        |
        |  group_lines(..., 5)
        v
outputs/sample-N.{sentences_hi,groups_hi,sentences_en,groups_en}.txt
```

A crucial engineering decision is that the three models are **never resident in memory at the same time**. `src/pipeline.py` loads stage 1, uses it for every file, closes it, loads stage 2, and so on. This is what makes the 1.5 GB budget plausible on a T4: the budget applies to the largest single stage, not to the sum of all three.

4. Vocabulary for the rest of the guide

Term	One-line meaning
Token	The unit a model reads: a word, a subword, a character, or a special symbol.
Tokenization	Splitting text into tokens.
Subword	A piece of a word, used so the model can handle rare words.
Embedding	A dense vector representing a token, word, or sentence.
Transformer	A neural architecture built from attention layers.
BERT	A transformer encoder pretrained to fill in masked words; good at understanding text.
mBERT	The multilingual version of BERT, pretrained on many languages including Hindi.

Term	One-line meaning
LoRA	Low-Rank Adaptation: train small matrices instead of all model weights.
Logit	The raw, unnormalized score a model produces before softmax.
Softmax	Turns logits into probabilities that sum to one.
Fine-tuning	Continuing to train a pretrained model on a task-specific dataset.
Epoch	One full pass over the training set.
Batch	A group of examples processed together.
Cross-entropy	The standard classification loss; punishes confident wrong answers.
Inference	Using a trained model to make predictions.
Decoder	The component that turns model scores into a final structure (a label, an order, a translation).
BLEU / chrF	Metrics for machine translation quality.
Kendall's tau	A rank-correlation metric for how close a recovered order is to the truth.

Part 0.5 — The approach we chose (read this first)

> **APPROACH CHOSEN — the thesis of the whole project.**
> We deliberately *decomposed* the assignment into three specialized stages instead of asking one model to do everything: a word-level punctuation labeller, a pairwise sentence-order scorer with an explicit decoder, and a dedicated Indic-to-English translator. The first two stages share one small pretrained backbone (mBERT) and are adapted with LoRA; the third is IndicTrans2 200M. There is no LLM, no fallback model, and no stage is allowed to rewrite the input words. The pipeline is engineered around the 1.5 GB budget by loading exactly one model at a time.

This chapter is intentionally placed before the theory. If you understand *why* this architecture was chosen, every later chapter has a place to hang. The rest of the guide explains the machinery; this chapter explains the design.

A.1 Why decompose the problem at all?

A single end-to-end model could, in principle, read the flat token stream and emit punctuated, ordered English. We rejected that for four reasons:

Reason	Explanation
Constraint fit	A single generative model large enough to do all three tasks well would not fit 1.5 GB and would look like an LLM, which the assignment forbids.
Evaluation	Decomposition gives one metric per stage: boundary F1, pair F1/Kendall's tau, BLEU/chrF. A single model gives one opaque score, and a failure is hard to localize.
Data availability	Wikipedia gives us punctuation labels and natural sentence order for free. It does not give us aligned triplets of (unpunctuated shuffled Hindi, punctuated ordered Hindi, English). Decomposition lets us train each stage on the supervision that actually exists.
Failure isolation	If ordering is weak, punctuation and translation can still be correct and measurable. If translation is unavailable, the first two stages still produce a valid partial result.

The decomposition is not an accident of convenience. It is the design that makes the constraints, the data, and the evaluation line up.

A.2 The eight principles behind the chosen approach

> **1. Preserve the words; predict structure.** Punctuation is a labelling problem, not a rewriting problem. Stage 1 attaches one of five labels to each existing word. This guarantees that the output words are the input words (minus `NULL`) and makes word-level evaluation exact. **2. Learn from natural text, not synthetic text.** The training corpus is Hindi Wikipedia. The punctuation and sentence order are already there; we only hide them to create supervision. This avoids hand-written rules and avoids LLM-generated data. **3. Split by article; shuffle inside windows.** The same article never appears in both training and test. Inside a punctuation window, sentences are shuffled to mimic the assignment input and to stop the model from leaning on cross-sentence continuity. **4. Reuse one backbone for two tasks.** mBERT is used once as a token classifier and once as a pair classifier. This keeps the dependency surface small, the memory profile predictable, and the story simple: one multilingual encoder, two heads. **5. Adapt cheaply with LoRA.** Full fine-tuning of 178M parameters is unnecessary and risky under the budget. LoRA trains about 0.33% of the parameters, keeps optimizer state tiny, and produces adapters small enough to save and share. **6. Make local decisions, then decode globally.** The ordering model answers only "does A come before B?". A separate decoder (Borda by default) turns all pairwise probabilities into a global permutation. This separates learning from combinatorial optimization and lets us compare decoders without retraining. **7. Use a dedicated translator, sentence by sentence.** IndicTrans2 is specialized for Indic languages and small enough for the budget. Translating sentence by sentence preserves exact alignment with the Hindi sentences and the groups of five. **8. Stage memory, fail loudly, report honestly.** Load one model, use it, free it. If a model or adapter is missing, raise a clear error; never silently switch to a fallback. Report what was actually run and distinguish it from what is merely documented.

A.3 The approach at a glance

Stage	Input	Chosen model	Chosen decoder	Why this choice
1. Punctuation	flat Hindi word list	mBERT + LoRA token classifier (5 labels)	argmax over triangular-weighted window probabilities	one decision per word; preserves words; cheap; strong sentence-ending cues
2. Ordering	list of plain Hindi sentences	mBERT + LoRA pair classifier (2 labels)	Borda over the full pairwise probability matrix	pairwise model generalizes to any length; Borda averages out local errors
3. Translation	ordered Hindi sentences	IndicTrans2 200M encoder-decoder	beam search (4 beams)	Indic-specialized, fits budget, sentence alignment is exact
Orchestration	seven raw files	none	load/free one stage at a time	keeps the largest resident model under 1.5 GB

The data flow is:

```
raw tokens -> [stage 1: label each word] -> sentences
           -> [stage 2: score all pairs + Borda] -> ordered sentences
           -> [stage 3: translate each sentence] -> aligned English
           -> group every five sentences
```

A.4 What we explicitly rejected

Rejected approach	Why it was rejected
Use an LLM to punctuate, reorder, and translate	Forbidden by the assignment; hides the pipeline; violates the memory and reproducibility spirit.
Full fine-tuning of mBERT	Roughly 178M parameters of gradients and AdamW state; unnecessary under the 1.5 GB budget; higher overfitting risk on 25k windows.
Sequence-to-sequence punctuation	Could rewrite words, is slower, and makes word-level evaluation harder. Token classification is the natural formulation.
A single end-to-end model for all three stages	No aligned training data; impossible to evaluate per stage; too large for the budget.
A direct permutation classifier for ordering	Output size varies with the number of sentences; hard to train; pairwise decisions generalize better.
Beam-only ordering	Locally greedy; measured worse than Borda (mean tau 0.0921 vs 0.2007).
TF-IDF-only ordering	Ignores narrative cues; measured tau 0.0631.
Document-level translation	Breaks sentence alignment and the required groups of five; can exceed the model context.
A fallback translation model	Hides failures and makes results incomparable; the project raises an error instead.
Keep all three models resident	Sum of the stages exceeds the budget; staging is simpler and safer than quantization.
Synthetic or LLM-generated training data	Forbidden and unnecessary; Wikipedia already contains the supervision we need.

A.5 How the approach satisfies the assignment constraints

Constraint	How the chosen approach satisfies it	Evidence
No LLM	Only mBERT and IndicTrans2 are used; the evaluation report sets `"no_llm": true`.	`src/constants.py:22–24`, `reports/evaluation.json`
1.5 GB GPU memory	LoRA, fp16 on CUDA, gradient checkpointing, one stage at a time.	`src/utils.py:74–99`, `src/pipeline.py:74–154`
Restore punctuation and boundaries	Five-label token classification with boundary reconstruction.	`src/punctuation/*`, boundary F1 0.8842
Recover sentence order	Pairwise classifier plus Borda/clusters/beam decoders.	`src/ordering/*`, Borda tau 0.2007
Translate Hindi to English	IndicTrans2 200M, sentence-level, aligned groups.	`src/translation/indictrans.py`, BLEU 34.125
Produce groups of five	`group_lines(..., 5)` applied to Hindi and English.	`src/utils.py:292–294`, `src/pipeline.py:115,146–149`
Reproducibility	Pinned dependencies, seeded splits, JSON reports, executed notebook.	`requirements-colab.txt`, `reports/`, `notebooks/pipeline_demo.ipynb`

A.6 Why this approach is defensible in a viva

- **Every design choice has a stated alternative.** The table above is the short version; Chapter 28 expands it.
- **Every stage has its own metric.** You can defend punctuation without defending ordering, and ordering without defending translation.
- **The weaknesses are measured, not hidden.** Comma F1, question/exclamation F1, and ordering tau are reported honestly.
- **The memory claim is architectural, not wishful.** The code physically releases each model before loading the next.
- **The data story is clean.** Natural Wikipedia supervision, article-level splits, and no synthetic labels.

> **If you remember one thing:** the project succeeds because it chooses the *right decomposition* for the constraints. mBERT labels words, a pairwise model ranks sentences, a decoder turns rankings into an order, and IndicTrans2 translates. Each stage is small, measurable, and replaceable.

Part I — NLP foundations from zero

5. What is natural language processing?

Natural language processing is the study of how to make computers do useful things with human language. "Useful things" includes classification (is this review positive?), extraction (who did what to whom?), generation (translate this sentence), retrieval (find the passage that answers this question), and structuring (turn a flat stream of words into sentences and paragraphs).

Language is hard for computers for reasons that are obvious to humans and invisible to code:

- **Ambiguity.** "I saw the man with the telescope" has two meanings. `बैंक` can mean a river bank or a financial bank.
- **Context dependence.** The meaning of a word depends on the sentence, the paragraph, and the world.
- **Compositionality.** The meaning of a sentence is built from its parts, but not by simple addition.
- **Variation.** Dialects, spelling variants, code-mixing, and noise.
- **Long-range structure.** A pronoun on page two may refer to a name on page one.

Classical NLP decomposed the problem into levels:

Level	Studies	Example
Phonology	Sounds	pronunciation, speech recognition
Morphology	Word structure	Hindi verb conjugation, plurals
Syntax	Sentence structure	subject-verb-object order
Semantics	Meaning	entailment, word sense
Pragmatics	Meaning in context	sarcasm, implicature
Discourse	Structure beyond a sentence	pronoun resolution, paragraph order

This project touches morphology (Hindi is agglutinative and has rich inflection), syntax (sentence boundaries depend on clause structure), semantics (ordering depends on meaning and discourse), and generation (translation). It does not attempt speech or dialogue.

5.1 Rule-based, statistical, neural, and LLM approaches

The history of NLP is a sequence of increasingly flexible representations:

1. **Rule-based.** Hand-written grammars and dictionaries. Predictable, explainable, brittle. The legacy `baseline/` folder is in this tradition: dynamic-programming segmentation using verb endings, sentence starters, `NULL` hints, and a length prior.
2. **Statistical.** Counts and probabilities learned from data: n-gram language models, hidden Markov models, TF-IDF plus logistic regression. The `lessons/train.py` exercise is a tiny example of this family.
3. **Neural.** Dense vectors and gradient-based learning: recurrent networks, convolutional networks, and then transformers. The project's two mBERT stages are here.

4. **Large language models.** Very large transformers trained on broad text, often used through prompting or instruction tuning. The assignment explicitly forbids this family, so the project does not use one.

> **Why forbid LLMs?** The assignment is designed to test whether you can build a *pipeline* from task-specific components: data preparation, token classification, pairwise ranking, and translation. Asking a chat model to "fix this text" would hide all of that. It would also make the memory and reproducibility claims meaningless.

5.2 Where this project sits

Assignment requirement	Technique used	Why
Restore punctuation and boundaries	token classification with mBERT + LoRA	one decision per word; preserves the original words; small and fast
Recover sentence order	pairwise classification plus a decoder	order is a global structure, but a pairwise model generalizes to any number of sentences
Translate	IndicTrans2 encoder-decoder	dedicated Hindi-to-English model, much smaller than an LLM, strong on Indic languages
Fit 1.5 GB	fp16 on CUDA, LoRA, one stage at a time	keeps the largest resident model under the cap

> **Check yourself.** Why is punctuation restoration formulated as token classification rather than as translation? *Because we do not want to rewrite the words; we want to attach one of five labels to each existing word. Translation-style generation would risk changing the text and would be far more expensive.*

6. From text to tokens

6.1 Text is Unicode, not bytes

A Python `str` is a sequence of Unicode code points. Each code point has a number. Hindi's Devanagari script occupies the range U+0900 to U+097F. Important characters in this project:

Character	Code point	Name	Role
`।`	U+0964	Devanagari danda	sentence end
`॥`	U+0965	Devanagari double danda	verse/section end; treated as sentence end here
`?`	U+003F	question mark	question
`!`	U+0021	exclamation mark	exclamation
`,`	U+002C	comma	comma

Devanagari is not a simple alphabet. Consonants have an inherent vowel, and vowel signs (matras) are separate combining characters that attach to the consonant. For example, `अयोध्या` is six code points:

```
अ  U+0905  DEVANAGARI LETTER A
य  U+092F  DEVANAGARI LETTER YA
ो  U+094B  DEVANAGARI VOWEL SIGN O      (combining)
ध  U+0927  DEVANAGARI LETTER DHA
्  U+094D  DEVANAGARI SIGN VIRAMA       (combining)
य  U+092F  DEVANAGARI LETTER YA
ा  U+093E  DEVANAGARI VOWEL SIGN AA      (combining)
```

The visual word is one grapheme cluster, but the string has multiple code points. This matters for any regex or tokenizer that assumes "one code point = one letter".

> **Common mistake.** Using `re.findall(r'\w+', text)` on Hindi. In Python, `\w` matches letters and digits but not all combining marks, so matras are silently dropped:

```
>>> import re
>>> re.findall(r'\w+', 'अयोध्या')
['अय', 'ध', 'य']
>>> re.findall(r'\w+', 'महालक्ष्मी')
['मह', 'लक', 'ष', 'म']
```

This is exactly why `src/data_prep.py` uses `str.split()` and `src/utils.py` implements `strip_punctuation()` without a word regex. Splitting on whitespace preserves every code point inside a token.

6.2 Word tokenization

Word tokenization splits a string into words. In English, whitespace plus punctuation rules are enough for many purposes; NLTK's `word_tokenize` additionally separates clitics and punctuation. In Hindi, whitespace is a strong signal because Hindi orthography separates words with spaces, but punctuation can be glued to the final word (`हुआ।`), and compounds may use hyphens (`राज्य–प्रबन्धन`).

The project's tokenizer is deliberately minimal:

```
raw_tokens = text.split()
words = [token for token in raw_tokens if token != 'NULL']
```

Then, during data preparation, `strip_punctuation()` separates a trailing mark from the word and records it as a label. This is a *reversible* tokenization: the punctuation is stored as a label rather than thrown away, and `restore_punctuation()` can put it back.

6.3 Sentence tokenization

Sentence splitting is harder. In English, a period can end a sentence or an abbreviation. In Hindi, the danda is a strong signal, but a danda can also appear in abbreviations or verse numbering. The project uses a regex lookbehind:

```
ARTICLE_PATTERN = re.compile(r'(?<=[।॥?!])\s+')
```

This splits at whitespace *after* a sentence-final mark. It is simple and works well for Wikipedia. It does not handle every edge case, which is acceptable because this is training-data extraction, not the final inference path. At inference time, sentence boundaries come from the trained classifier, not from this regex.

6.4 Subword tokenization

Modern transformer models rarely use whole words. A vocabulary of all Hindi words would be enormous and would still miss new words. Instead, models use **subword tokenization**: frequent words stay whole, rare words are split into smaller pieces.

The three main algorithms:

- **BPE (Byte Pair Encoding).** Start with characters (or bytes). Repeatedly merge the most frequent adjacent pair into a new symbol. This builds a vocabulary of frequent pieces.
- **WordPiece.** Similar in spirit, but merges are chosen by a likelihood criterion rather than raw frequency. Continuation pieces are marked with `##`. This is what BERT and mBERT use.
- **Unigram / SentencePiece.** Start with many candidate pieces and prune the vocabulary probabilistically. SentencePiece treats the input as a raw character stream, which is convenient for languages without spaces. IndicTrans2 uses a SentencePiece-style vocabulary.

A useful mental model: subword tokenization is a compression scheme learned from data. Common strings become single tokens; rare strings become several tokens. The model never sees an "unknown word" in the classical sense; it sees pieces.

A real mBERT example

The sentence `भारत एक विशाल देश है।` becomes:

```
[CLS] भारत एक वि ##शा ##ल देश है । [SEP]
```

The word `विशाल` is split into `वि`, `##शा`, `##ल`; the `##` means "this piece continues the previous word". Other examples:

Word	mBERT pieces
`अयोध्या`	`अ`, `##य`, `##ोध`, `##्या`
`राज्य-प्रबन्धन`	`राज्य`, `-`, `प`, `##्र`, `##बन्ध`, `##न`
`महालक्ष्मी`	`म`, `##हा`, `##ल`, `##क्ष`, `##्मी`
`राम`	`राम` (one piece)

> **Why this matters for labelling.** The punctuation label belongs to a *word*, but the model predicts a label for each *subword*. The project's dataset puts the word's label on the first subword and sets every continuation subword to `-100`, which the loss ignores. That way each word is supervised exactly once. See `src/punctuation/dataset.py:24-43`.

Special tokens, padding, truncation, attention masks

- `[CLS]` is a special classification token at the start. BERT's final hidden state for `[CLS]` is used as a sentence-level summary in sequence classification.
- `[SEP]` separates sequences. For a pair, the format is `[CLS] A [SEP] B [SEP]`.
- `[PAD]` fills shorter sequences in a batch so they have equal length.
- **Attention mask** is 1 for real tokens and 0 for padding, so the model ignores padding.
- **Truncation** cuts a sequence longer than the model's maximum length (128 for punctuation windows, 192 for sentence pairs).
- `word_ids()` maps each subword back to its original word index, with `None` for special tokens. This is the bridge between word-level labels and subword-level predictions.

```
encoding = tokenizer(words, is_split_into_words=True, truncation=True,
                     max_length=128)
word_ids = encoding.word_ids()   # e.g. [None, 0, 0, 1, 1, 1, 2, None]
```

> **Check yourself.** Why can't we simply label every subword with the word's punctuation label? *Because then a five-subword word would contribute five identical training signals, and at inference time the same word would produce five possibly conflicting predictions. First-subword labelling gives one prediction per word and keeps the loss balanced.*

7. Embeddings and vector representations

A model cannot do arithmetic on the string `भारत`. It needs numbers. An **embedding** is a dense vector that represents a token, word, sentence, or document.

> **APPROACH CHOSEN.** We do not train embeddings from scratch. We reuse mBERT's contextual embeddings, and we use TF-IDF only where a cheap lexical representation is enough (the ordering ablation and the clustering decoder). This keeps the learned representation strong and the extra machinery minimal.

7.1 One-hot and bag-of-words

The simplest representation is a vector with one dimension per vocabulary item. For a 100,000-word vocabulary, `भारत` might be a vector of 99,999 zeros and one one. This is interpretable but enormous, sparse, and semantically empty: `भारत` and `भारतीय` are as far apart as `भारत` and `banana`.

A **bag of words** counts word occurrences in a document and ignores order. It is the basis of TF-IDF.

7.2 TF-IDF

TF-IDF weights a term by how often it appears in a document, discounted by how common it is across all documents:

```
tf-idf(t, d) = tf(t, d) * log(N / df(t))
```

- `tf(t,d)` is the frequency of term `t` in document `d`.
- `df(t)` is the number of documents containing `t`.
- `N` is the number of documents.

A term that appears everywhere (like "the") gets a low weight; a term that is rare but appears in this document gets a high weight. The project uses TF-IDF in two places:

- **Ordering ablation:** a TF-IDF similarity matrix plus a greedy order, with no mBERT. This is the "can simple lexical similarity do the job?" baseline.
- **Clustering decoder:** character n-gram TF-IDF (`analyzer='char_wb'`, 2- to 4-grams) plus KMeans to group topically similar sentences before ordering.

Character n-grams are useful for Hindi because they capture morphology and tolerate spelling variation better than whole-word features. `char_wb` means n-grams are built within word boundaries, with padding at the edges.

7.3 Static word embeddings

Word2vec, GloVe, and fastText learn a vector per word from co-occurrence statistics:

- **Word2vec (skip-gram).** Predict context words from a target word. The learned input vectors become the embeddings. The objective for a word `w_t` and context window `c` is to maximize the sum of `log p(w_{t+j} | w_t)` over nearby context positions.
- **GloVe.** Factorizes a global co-occurrence matrix so that vector differences encode meaning relations.
- **fastText.** Represents a word as the sum of its character n-gram vectors, so it can handle unseen words and morphology.

These embeddings are **static**: the vector for `बैंक` is the same in every sentence. That is a fundamental limitation.

7.4 Contextual embeddings

Transformers produce **contextual** representations. The vector for a token is computed from the token plus its entire context, layer by layer. In BERT, the final hidden state for each token is a 768-dimensional vector that depends on the sentence. The same word in two different sentences can have different vectors.

For a sentence-level task, BERT's `[CLS]` vector is a learned summary of the whole sequence. For a token-level task, each token's final hidden state is passed through a classification head.

> **Choice:** mBERT contextual embeddings for both learned stages, not static word vectors. **Reason:** punctuation depends on syntax and local context; ordering depends on the relationship between two sentences. Static vectors cannot represent either. **Cost:** a 178M-parameter backbone, which we fit with LoRA and fp16.

7.5 Cosine similarity

To compare two vectors, the standard measure is cosine similarity:

```
cos(u, v) = (u . v) / (||u|| * ||v||)
```

It measures the angle between vectors and ignores magnitude. It is used in the clustering decoder to choose a medoid (the member closest to the cluster centroid) and in the TF-IDF ablation.

> **Check yourself.** Why does the project use contextual mBERT embeddings for ordering but TF-IDF for clustering? *TF-IDF is cheap and good enough to group topically similar sentences; the expensive mBERT pair model is reserved for the finer before/after decisions. The evaluation compares this hybrid against a TF-IDF-only ablation.*

8. Transformers and BERT

> **APPROACH CHOSEN.** Both learned stages use the same encoder-only backbone, mBERT, with a small task-specific head. Stage 1 uses token classification; stage 2 uses sequence classification. The translation stage is the only encoder-decoder. We deliberately avoid a single multi-task model because separate heads are easier to train, evaluate, and reason about.

8.1 The basic neural network

A neural network is a stack of layers that transform vectors. A linear layer computes `y = Wx + b`. A nonlinearity such as ReLU or GELU lets the network learn curved decision boundaries. A loss function measures how wrong the output is; backpropagation computes gradients; an optimizer updates the weights.

For classification, the model produces **logits** `z`, one per class. **Softmax** turns them into probabilities:

```
p_i = exp(z_i) / sum_j exp(z_j)
```

The **cross-entropy loss** for the correct class `y` is `-log(p_y)`. If the model is confident and correct, the loss is near zero. If it is confident and wrong, the loss is large. This is the loss used in both mBERT stages (with class weights in punctuation).

8.2 Attention

The central operation of a transformer is **scaled dot-product attention**:

```
Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V
```

- `Q` (query), `K` (key), and `V` (value) are linear projections of the input.
- `Q K^T` computes a score for every pair of positions: how much should position `i` attend to position `j`?
- Dividing by `sqrt(d_k)` keeps the scores from growing too large as the dimension grows.
- Softmax turns the scores into attention weights.
- Multiplying by `V` produces a weighted sum of value vectors.

Multi-head attention runs several attention operations in parallel with different projections, then concatenates and projects the results. Different heads can learn different relationships: syntax, coreference, local order, and so on.

A transformer block adds:

- **Residual connections:** `x + f(x)`, which make deep networks trainable.
- **Layer normalization:** stabilizes activations.
- **Feed-forward network:** a two-layer MLP applied to each position independently, usually with a hidden size 4 times the model dimension.
- **Dropout:** randomly zeroes activations during training to reduce overfitting.

8.3 Encoder, decoder, and sequence-to-sequence

- An **encoder** reads the whole input and produces contextual representations. BERT is an encoder.
- A **decoder** generates output one token at a time, attending to what it has already generated and, in a sequence-to-sequence model, to the encoder's output.
- A **sequence-to-sequence** model combines both. IndicTrans2 is an encoder-decoder: the encoder reads Hindi, the decoder generates English.

BERT uses **bidirectional** attention: every token can attend to every other token. This is great for understanding. It is not directly suited to generation, because generation must not see the future. GPT-style models use **causal** attention: each token can attend only to earlier tokens.

8.4 BERT and mBERT

BERT is pretrained with two objectives:

1. **Masked language modelling (MLM).** Randomly mask some tokens and predict them from context. This forces the model to learn syntax and semantics.
2. **Next sentence prediction (NSP).** Predict whether sentence B follows sentence A. NSP is less important in modern practice, but it is part of the original BERT.

`bert-base-multilingual-cased` is the multilingual version, trained on Wikipedia in over 100 languages. Its configuration:

Property	Value
Vocabulary size	119,547
Hidden size	768
Layers	12
Attention heads	12
Feed-forward size	3,072
Maximum positions	512
Parameters	about 178 million

It is **cased**, so it distinguishes uppercase and lowercase; this matters little for Hindi but is the standard checkpoint. It has no task-specific head until we add one.

8.5 Task heads

A pretrained encoder is a backbone. To use it, we attach a **head**:

- **Token classification:** a linear layer maps each token's 768-dimensional hidden state to 5 logits (one per punctuation label).
- **Sequence classification:** the `[CLS]` vector is mapped to 2 logits (wrong order / correct order).
- **Sequence-to-sequence:** the encoder-decoder generates token by token; no simple classification head.

The head is randomly initialized and trained from scratch. This is why the notebook prints "Some weights of BertForTokenClassification were not initialized from the model checkpoint" when loading the base model. That warning is expected: the backbone is pretrained, the head is new.

8.6 Why mBERT for this project

- It is multilingual and has seen Hindi during pretraining.
- It is small enough (about 178M parameters) to fit the budget with LoRA and fp16.
- It is widely available, well documented, and works with the pinned `transformers` version.
- It supports both token classification and sequence classification with the same backbone.

Alternatives such as IndicBERT or MuRIL are also reasonable; mBERT was chosen for availability and because it is the model used in the course notes. The guide's design-decisions chapter discusses the trade-offs.

> **Check yourself.** Why does the ordering model use the `[CLS]` vector rather than averaging all token vectors? *The `[CLS]` representation is trained end-to-end for the pair classification objective, so it learns which aspects of the two sentences matter for their relative order. Averaging is a fixed, weaker pooling operation.*

9. Transfer learning, fine-tuning, and LoRA

> **APPROACH CHOSEN.** Freeze the mBERT backbone and train LoRA adapters on the query and value projections, plus the full classification head. This is the single most important memory decision in the project. It gives two task-specific models for roughly the cost of one backbone, and the adapters are small enough to save and reload.

9.1 Pretraining and fine-tuning

Pretraining teaches a model general language structure on a large corpus. **Fine-tuning** continues training on a smaller, task-specific dataset. The pretrained weights give the model a strong starting point, so fine-tuning needs much less data and time than training from scratch.

Full fine-tuning updates every weight. For a 178M-parameter model, that means storing gradients for all 178M parameters, storing optimizer state (AdamW keeps two moments per parameter), and accepting a large memory footprint and a risk of catastrophic forgetting.

The assignment's 1.5 GB budget makes full fine-tuning unattractive.

9.2 Parameter-efficient fine-tuning

Parameter-efficient fine-tuning (PEFT) freezes most of the pretrained model and trains a small number of new parameters. The most popular method is **LoRA (Low-Rank Adaptation)**.

9.3 The LoRA idea

A linear layer computes `y = Wx`. Full fine-tuning learns a new `W'`. LoRA instead freezes `W` and learns a low-rank update:

```
W' = W + (alpha / r) * B A
```

where:

- `W` is the frozen pretrained matrix, shape `d_out x d_in`,
- `A` is a small matrix of shape `r x d_in`,
- `B` is a small matrix of shape `d_out x r`,
- `r` is the rank (16 in this project),
- `alpha` is a scaling constant (32 here).

The number of trainable parameters per adapted matrix is `r(d_in + d_out)`, which is tiny compared with `d_in * d_out`.

For mBERT, `d = 768` and `r = 16`:

```
per matrix = 16 * (768 + 768) = 24,576
```

The project adapts the **query** and **value** projections in all 12 layers. That is 24 matrices:

```
24 * 24,576 = 589,824
```

Add the fully trained classification head:

- punctuation: `768 * 5 + 5 = 3,845`
- ordering: `768 * 2 + 2 = 1,538`

So:

Stage	LoRA params	Head params	Trainable total	Total model	Trainable fraction
Punctuation	589,824	3,845	593,669	177,860,362	0.334%
Ordering	589,824	1,538	591,362	about 178.4M	0.331%

> **Why query and value?** Adapting `Q` and `V` changes what each token looks for and what it passes on, which is a strong lever for representation learning. Adapting the output projection is also common; adapting every matrix costs more. The choice is a standard, effective default.

9.4 LoRA hyperparameters

Parameter	Value	Meaning
`r`	16	rank of the update; higher = more capacity, more parameters
`lora_alpha`	32	scaling; effective scale is alpha/r = 2
`lora_dropout`	0.1	dropout on the LoRA path
`target_modules`	`['query', 'value']`	which linear layers get adapters
`modules_to_save`	`['classifier']`	head trained and saved in full
`bias`	`'none'`	no bias training

The adapter files are tiny. `adapter_model.safetensors` contains the LoRA matrices and the classifier head; the base model is loaded separately. At inference time, `PeftModel.from_pretrained(base_model, adapter_dir)` reconstructs the adapted model.

> **Important nuance.** LoRA does not reduce the memory needed to *store the base model*. It reduces the memory needed to *train*. At inference, the base weights are still there. The project's savings come from fp16 and from loading one model at a time, not from LoRA alone.

9.5 Adapter merging

LoRA updates can be merged into the base weights for deployment: `W' = W + (alpha/r) B A`. After merging, inference has no LoRA overhead. This project keeps the adapter separate for clarity and because the merged base would be large. IndicTrans2, by contrast, is used as a full model (no LoRA).

> **Check yourself.** If LoRA rank is increased from 16 to 64, what happens to the number of trainable parameters and to overfitting risk? *Parameters grow roughly linearly in `r` (4x), capacity grows, and overfitting risk grows, especially with only 4,000 training examples.*

10. Training mechanics

> **APPROACH CHOSEN.** Train each stage independently with a standard Hugging Face `Trainer`, a fixed seed, article-level splits, and a task-appropriate validation metric. Punctuation uses class weights and boundary F1; ordering uses balanced pairs and accuracy. Everything else is deliberately conventional so the unusual parts (LoRA, staging, Devanagari handling) stand out.

10.1 Data splits

The project splits **by article**, not by window or sentence:

- 2,700 articles for training,
- 150 for validation,
- 150 for test.

Splitting by article prevents leakage: two windows from the same article, or two sentence pairs from the same article, can be highly correlated. If one appeared in training and the other in test, the test score would be optimistic. The same article IDs define both the punctuation split and the ordering split, so the two stages are evaluated on the same held-out articles.

The punctuation split produces:

Split	Articles	Windows	Words	none	comma	sentence_end	question	exclamation
train	2,700	25,000	1,817,660	90.95%	3.26%	5.77%	0.013%	0.008%
val	150	2,317	168,661	91.38%	2.99%	5.60%	0.009%	0.021%
test	150	2,208	159,105	90.90%	3.20%	5.88%	0.018%	0.014%

The ordering split produces 40,000 training pairs, 3,000 validation pairs, and 3,000 test pairs, plus 40 held-out full articles for Kendall's tau.

10.2 Batch, step, epoch

- **Batch size** is the number of examples processed before one optimizer update.
- **Step** is one optimizer update.
- **Epoch** is one full pass over the training set.

With 25,000 windows, batch size 8, and gradient accumulation 2, one epoch has about `ceil(25,000 / (8 * 2)) = 1,563` optimizer steps. The effective batch size is 16 examples per update.

10.3 Learning rate, warmup, weight decay

- **Learning rate (LR)** controls step size. Too high diverges; too low learns slowly.
- **Warmup** gradually increases the LR from near zero to the target over the first 6% of steps, which stabilizes early training.
- **Weight decay** penalizes large weights (a regularizer). The project uses 0.01.
- **AdamW** is the optimizer: Adam with decoupled weight decay. It maintains a moving average of gradients and squared gradients per parameter.

10.4 Gradient accumulation and checkpointing

- **Gradient accumulation** sums gradients over several mini-batches before stepping. It simulates a larger batch without the memory cost.
- **Gradient checkpointing** stores fewer activations during the forward pass and recomputes them during the backward pass. It trades compute for memory. This is essential for fitting mBERT in a small budget.

10.5 Mixed precision

fp16 stores activations and weights in 16-bit floats, roughly halving memory and speeding up math on supported GPUs. The project enables fp16 only on CUDA:

```
fp16=torch.cuda.is_available()
```

On Apple MPS, training runs in fp32 because this torch build's fp16/MPS path is not reliable. Inference uses fp16 on CUDA and fp32 otherwise.

10.6 Seeds and reproducibility

`set_seed(42)` seeds Python, NumPy, and PyTorch. The data preparation uses seeded `random.Random` instances for shuffling and splitting. Exact reproducibility across hardware is never guaranteed (different GPUs, different kernels, nondeterministic operations), but the same machine and seed give stable results.

10.7 Class weights

The punctuation dataset is extremely imbalanced: 90.9% of words take no mark. A plain cross-entropy loss would learn to predict `none` almost always and still achieve high accuracy. The project uses inverse-frequency class weights:

```
w_c = N / (K * n_c)
```

where `N` is the total number of labels, `K = 5` is the number of classes, and `n_c` is the count of class `c`. Weights are capped at 20 to avoid instability.

Using the training counts:

Class	Count	Raw weight	Capped weight
none	1,653,107	0.220	0.220
comma	59,324	6.128	6.128
sentence_end	104,840	3.467	3.467
question	236	1,540.4	20.0
exclamation	153	2,375.3	20.0

The cap prevents the two rare classes from dominating the loss. It does not fully solve the rarity problem; see the results chapter.

`WeightedTrainer` overrides `compute_loss` to use `CrossEntropyLoss(weight=...)` with `ignore_index=-100`, so continuation subwords do not contribute.

10.8 Model selection

At the end of each epoch, the trainer evaluates on the validation set. It saves the checkpoint with the best validation metric:

- punctuation: best **boundary F1** (not accuracy, because accuracy is dominated by `none`),
- ordering: best **accuracy** (the classes are balanced, so accuracy is meaningful).

`load_best_model_at_end=True` restores the best checkpoint after training. `save_total_limit=1` keeps only the most recent checkpoint to save disk.

10.9 Exact training settings

Setting	Punctuation	Ordering
Base model	`bert-base-multilingual-cased`	`bert-base-multilingual-cased`
Task	token classification (5 labels)	sequence classification (2 labels)
LoRA `r` / `alpha` / dropout	16 / 32 / 0.1	16 / 32 / 0.1
LoRA targets	query, value	query, value
Head saved	classifier	classifier
Default epochs	3	2
Default batch size	8	8
Effective batch	16 (accum 2)	16 (accum 2)
Learning rate	3e-4	2e-4
Weight decay	0.01	0.01
Warmup	6%	6%
Max sequence length	128	192
fp16	CUDA only	CUDA only
Gradient checkpointing	yes	yes
Best metric	boundary F1	accuracy
Seed	42	42

The locally verified adapters used different sample counts and epochs because MPS is slow:

Setting	Punctuation (local)	Ordering (local)
Training examples	4,000 windows	4,000 pairs
Epochs	2	1
Batch size	16	16
Learning rate	3e-4	2e-4
Test boundary F1	0.8633	—
Test macro F1	—	0.6534
Trainable parameters	593,669	591,362
Total parameters	177,860,362	about 178.4M

> **Be precise in the viva.** The reported pipeline scores (boundary F1 0.8842, ordering tau 0.2007) come from these subset adapters, not from a full-data run. The full-data recipe is in the notebook and README; it has not been executed to completion in this repository. Saying otherwise would be a factual error.

10.10 Diagnosing training

Signals to watch:

- **Loss goes down, validation F1 goes up:** learning.
- **Training loss goes down, validation loss goes up:** overfitting; reduce epochs, increase dropout, add data.
- **Both losses flat:** learning rate too low, or labels misaligned.
- **Loss becomes NaN:** learning rate too high, fp16 instability, or bad class weights.
- **Accuracy high but F1 low:** class imbalance; look at per-class scores.
- **Boundary F1 high but comma F1 low:** the model is good at sentence endings but weak on commas; inspect comma examples.

The local training summaries show healthy learning: punctuation eval loss 0.4399, boundary F1 0.8246 after 2 epochs; ordering eval loss 0.5827, macro F1 0.6768 after 1 epoch. The ordering model is not overfit after one epoch, which is why the full recipe uses two.

> **Check yourself.** Why is boundary F1 a better model-selection metric than token accuracy for punctuation? *Because 90.9% of tokens are* `none`*; a model that predicts* `none` *everywhere gets 90.9% accuracy but finds zero boundaries. Boundary F1 forces the model to actually detect sentence endings.*

11. Inference and decoding

> **APPROACH CHOSEN.** Keep the inference path as close as possible to the training distribution, then add only the minimum decoding logic needed. Punctuation uses overlapping windows with triangular weighting; ordering uses an explicit global decoder; translation uses standard beam search. No stage relies on an LLM or a hidden heuristic.

11.1 Argmax and softmax

For a classification model, the simplest decoder is **argmax**: choose the class with the highest logit. Equivalently, choose the highest softmax probability. The punctuation model uses argmax after averaging probabilities across windows.

11.2 Sliding windows and triangular weighting

A document can be longer than the model's maximum input length. The punctuation model uses 64-word windows with stride 40. A word near the edge of a window has less context on one side, so its prediction is less reliable. The project averages predictions with a **triangular weight**:

```
w(p, L) = max(0.05, 1 − |p − c| / (c + 1)),  c = (L − 1) / 2
```

where `p` is the position in the window and `L` is the window length. Words in the middle get weight near 1; words at the edges get weight near 0.05. Predictions are accumulated as `score[word] += weight * probability` and divided by the total weight.

Why this works: every word is seen in multiple windows (except near document edges), and the windows where it is central dominate. The result is smoother and more accurate than using a single window.

11.3 Batching

Inference processes windows in batches (16 for punctuation, 32 pairs for ordering, 4 sentences for translation). Batching improves GPU utilization. The code sorts nothing; it simply chunks the list.

11.4 Beam search

For sequence generation, greedy decoding picks the most likely next token at each step. **Beam search** keeps the `k` most likely partial sequences (the beam) and expands them, which can find a better global sequence. IndicTrans2 uses `num_beams=4`, `max_length=256`, and `early_stopping=True`.

Beam search is not used for punctuation or pair classification because those are single-step decisions, not sequences.

11.5 Ordering decoders

The ordering model produces a matrix `M` where `M[i,j] = P(sentence i before j)`. Three decoders turn this matrix into a permutation:

- **Borda (default):** score each sentence by its mean probability of coming before all others, then sort descending.
- **Beam:** build a chain, keeping the best partial chains, scoring each adjacent pair with `M`.
- **Clusters:** TF-IDF + KMeans to group sentences, order cluster medoids with the pair model, then beam-search inside each cluster.

The evaluation compares all three plus a TF-IDF-only ablation. Borda wins on mean Kendall's tau.

11.6 Complexity

For `n` sentences, the pair model evaluates `n(n−1)/2` pairs. For 99 sentences that is 4,851 pairs. This quadratic cost is why ordering is the slowest stage in the pipeline (130.53 seconds for all seven files on MPS). Clustering reduces the number of expensive pairs inside clusters, but the project still computes the full matrix for evaluation.

> **Check yourself.** Why is `M[j,i] = 1 − M[i,j]` valid? *The classifier is trained on both orders of every pair, so it approximates a symmetric preference. The code explicitly fills the transpose as the complement to keep the matrix consistent and avoid a second model call.*

12. Metrics: what the numbers mean

> **APPROACH CHOSEN.** Report the metric that matches the decision being made, not a single headline number. Boundary F1 for punctuation (not accuracy), pair F1 plus Kendall's tau for ordering (not pair accuracy alone), and BLEU plus chrF for translation (not one metric).

12.1 Confusion matrix, accuracy, precision, recall, F1

For binary decisions, define:

- **True positive (TP):** predicted positive, actually positive.
- **False positive (FP):** predicted positive, actually negative.
- **True negative (TN):** predicted negative, actually negative.
- **False negative (FN):** predicted negative, actually positive.

Then:

```
accuracy  = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall    = TP / (TP + FN)
F1        = 2 * precision * recall / (precision + recall)
```

Accuracy is intuitive but misleading under imbalance. Precision asks "when the model says positive, how often is it right?" Recall asks "of all real positives, how many did the model find?" F1 balances the two.

12.2 Macro, micro, weighted

- **Macro average:** compute the metric per class and average, giving each class equal weight. Good when rare classes matter.
- **Micro average:** pool all decisions and compute globally; dominated by frequent classes.
- **Weighted average:** average per-class metrics weighted by class support.

The ordering pair metrics use macro averaging because the two classes are balanced but we still want a symmetric view. The punctuation per-class F1 is reported class by class, which is more informative than any single average.

12.3 Boundary F1

The punctuation model predicts five labels. For the assignment, what matters most is whether a word is a **sentence boundary**. Define:

- boundary = label in `{sentence_end, question, exclamation}`.
- non-boundary = `{none, comma}`.

Then boundary precision/recall/F1 treat boundary detection as a binary problem. This collapses the rare question/exclamation classes into the boundary class, which is useful because those classes are too rare to evaluate separately. It still does not fix the fact that the model rarely predicts them.

12.4 Per-class F1 and the accuracy trap

The reported punctuation results:

Metric	Value
Token accuracy	0.8724
Boundary precision	0.8539

Metric	Value
Boundary recall	0.9167
Boundary F1	0.8842
F1 `none`	0.9285
F1 `comma`	0.3161
F1 `sentence_end`	0.8838
F1 `question`	0.0
F1 `exclamation`	0.0

The accuracy of 87.24% sounds high, but a trivial "always `none`" model would get about 90.9% on this distribution. The meaningful number is boundary F1 0.8842: the model finds sentence endings well (sentence-end F1 0.8838) but is weak on commas (0.3161) and never predicts question or exclamation marks. This is a direct consequence of the training distribution.

> **Do not compare training-time and pipeline-time punctuation scores as if they were the same experiment.** The training summary's test boundary F1 (0.8633) is computed with a single truncated window per example, labels on first subwords only. The evaluation's 0.8842 uses sliding-window inference with triangular weighting on 1,500 test windows. Different inference procedures produce different numbers.

12.5 Pair accuracy and macro F1 for ordering

The ordering test set has 3,000 balanced pairs. Results:

Metric	Value
Pair accuracy	0.6543
Macro precision	0.6560
Macro recall	0.6543
Macro F1	0.6534

A random classifier gets 0.5. The model is clearly learning, but the margin is modest. Pairwise accuracy is not the same as recovering a full order: a model can be right on many pairs and still produce a poor global order because errors interact.

12.6 Kendall's tau

Kendall's tau measures rank correlation between the recovered order and the true order. For every pair of items, count whether their relative order agrees or disagrees:

```
tau = (concordant − discordant) / (concordant + discordant)
```

- `tau = 1`: perfect order.

- `tau = 0`: no better than random.
- `tau = −1`: exactly reversed.

The evaluation shuffles each held-out article with a fixed seed, recovers an order, and computes tau against the original order. Results on 40 articles:

Decoder	Mean tau	Median tau	Min	Max
Borda	0.2007	0.2304	-0.1775	0.6444
Clusters	0.1220	0.1081	-0.2652	0.7143
Beam	0.0921	0.0693	-0.2467	0.7143
TF-IDF ablation	0.0631	0.0589	-0.4847	0.5714

Borda is the best on average, which justifies making it the default. Some articles have negative tau, meaning the recovered order is worse than random for that article. This happens when the pairwise model is confidently wrong about a topic shift. Max tau around 0.64-0.71 shows that some articles are recovered reasonably well.

12.7 BLEU

BLEU measures n-gram overlap between a hypothesis translation and one or more references, with a brevity penalty:

```
BLEU = BP * exp(sum_{n=1..N} w_n * log(p_n))
```

- `p_n` is the clipped n-gram precision: the fraction of hypothesis n-grams that appear in the reference, with counts clipped to the reference count.
- `BP = min(1, exp(1 - r/c))` where `r` is reference length and `c` is hypothesis length; it penalizes short outputs.
- Usually `N = 4` and weights are uniform.

BLEU rewards exact phrasing and is sensitive to word order. It correlates imperfectly with human judgment, especially for paraphrases.

12.8 chrF

chrF computes character n-gram precision and recall and combines them with an F-score. It is more tolerant of morphology and paraphrase than BLEU, which is useful for Hindi-English. The project uses `CHRF(word_order=2)`, which also considers word bigrams.

12.9 Translation results

On 100 IITB Hindi-English test sentences:

Metric	Value
BLEU	34.125
chrF	59.686

These are respectable for a 200M-parameter model on a general benchmark. The sample translations show good meaning preservation with some paraphrase and occasional literal phrasing. BLEU and chrF are corpus-level scores; individual sentences can differ a lot.

> **Check yourself.** A model gets 95% token accuracy on punctuation but boundary F1 0.2. What is happening? *It is predicting the majority class almost everywhere. Accuracy is high because most words take no mark, but it misses most boundaries. This is exactly why the project reports boundary F1 and per-class F1.*

13. Data ethics, licensing, and responsible claims

> **APPROACH CHOSEN.** Use public, natural data; respect gated-model terms; keep all computation local; and state clearly what was and was not executed. The project's credibility rests on not overclaiming.

13.1 Data source

Training data comes from Hindi Wikipedia (`wikimedia/wikipedia`, config `20231101.hi`). Wikipedia text is publicly available under CC BY-SA and GFDL. The project uses it to learn punctuation and sentence order. It does not redistribute the raw dump; it stores derived datasets locally and ignores them in git.

The evaluation uses `cfilt/iitb-english-hindi` for BLEU/chrF. Check the dataset card for its license and terms before redistribution.

13.2 Model licenses and gating

- `bert-base-multilingual-cased` is publicly available on Hugging Face.
- `ai4bharat/indictrans2-indic-en-dist-200M` is **gated**: you must accept its terms on Hugging Face and authenticate before downloading. The local verification used an ungated merged mirror because the gated repository returned 401 without a token.
- Always check the model card for the license and intended use. A model's availability does not automatically mean unrestricted commercial use.

13.3 Privacy

The inputs are assignment texts, not personal user data. The pipeline does not log text to external services, does not call an API, and does not send data to an LLM. All computation is local or on the user's Colab runtime. This is a strong privacy property compared with a hosted chat model.

13.4 Bias and failure modes

Wikipedia is not a neutral sample of Hindi. It over-represents formal, encyclopedic text and under-represents dialogue, social media, and dialect. The model therefore:

- handles encyclopedic prose better than conversation,
- rarely predicts question/exclamation marks because Wikipedia uses them rarely,
- may impose a formal register on informal text.

Translation models can also reflect gender and cultural biases in their training data. The project does not attempt to measure or mitigate these biases; it is a course assignment, not a production system.

13.5 Honest reporting

The guide and README distinguish what was actually run locally (subset adapters, MPS), what is documented but not run (full-data Colab training), and what is skipped gracefully (translation evaluation if the dataset is unavailable). This honesty matters. A viva examiner will respect "we trained on 4,000 examples and here is the measured effect" far more than an inflated claim that cannot be reproduced.

> **Check yourself.** Why is "no LLM" both a technical and an ethical claim? *Technically, it means every output comes from a small, inspectable model. Ethically, it means no data leaves the machine and no opaque hosted service influences the result.*

14. The end-to-end mental model

Before diving into code, hold this picture in your head:

1. **Words in, labels out.** Stage 1 does not rewrite words. It attaches one of five labels to each word. The words are preserved exactly (after `NULL` removal).
2. **Labels define sentences.** A boundary label closes a sentence. The sentence text is the words plus their predicted marks.
3. **Sentences are nodes.** Stage 2 treats each sentence as a node and asks a pairwise question about every pair. The output is a matrix of probabilities, not an order.
4. **A decoder turns the matrix into an order.** This is a separate algorithmic step; the model does not directly output a permutation.
5. **Translation is sentence-level.** Each ordered sentence is translated independently. Grouping happens after translation, by position.
6. **Memory is managed by staging.** Load, use, free, repeat. The budget applies to the largest stage.
7. **Every stage has a metric.** Token/boundary F1 for punctuation, pair F1 and Kendall's tau for ordering, BLEU/chrF for translation.

If you can explain these seven points and the data shapes, you can explain the project.

Part II — The project in depth

15. Repository tour

```
nlp-zero-to-hero/
|-- data/
|   |-- raw/                    # sample-1..7.txt (space-separated words, NULL tokens)
|   `-- processed/              # generated JSONL datasets (git-ignored)
|-- src/
|   |-- constants.py            # paths, model ids, labels, hyper-parameters
|   |-- utils.py                # seeding, devices, memory, Devanagari helpers
|   |-- data_prep.py            # Wikipedia -> punctuation + ordering datasets
|   |-- punctuation/
|   |   |-- dataset.py          # word -> subword label alignment
|   |   |-- train.py            # mBERT + LoRA token classifier
|   |   `-- predict.py          # sliding-window inference
|   |-- ordering/
|   |   |-- pairs.py            # before/after pair dataset
|   |   |-- train.py            # mBERT + LoRA pair classifier
|   |   `-- decode.py           # Borda, clusters, beam, Kendall's tau
|   |-- translation/
|   |   `-- indictrans.py       # IndicTrans2 wrapper
|   |-- pipeline.py             # end-to-end orchestration
|   `-- evaluate.py             # metrics and reports
|-- notebooks/
|   |-- colab_end_to_end.ipynb  # 28-cell full Colab recipe
|   |-- pipeline_demo.ipynb     # 13-cell executed local demo
|   `-- lesson_01..03.ipynb     # early generic NLP exercises
|-- baseline/                   # legacy rule-based version (comparison only)
|-- lessons/                    # preprocessing and TF-IDF exercises
|-- outputs/                    # generated Hindi/English sentence and group files
|-- reports/                    # JSON metrics
|-- scripts/colab_setup.sh      # dependency installer
|-- requirements-colab.txt      # verified GPU/Colab pins
|-- requirements.txt            # local CPU environment
`-- README.md
```

15.1 What each source file does

File	Responsibility	Key detail
`src/constants.py`	Single source of truth for paths, model names, labels, hyperparameters	all paths are absolute, derived from the repo root

File	Responsibility	Key detail
`src/utils.py`	Reusable helpers	`set_seed`, `get_device`, `attention_implementation`, `limit_gpu_memory`, `sliding_windows`, `window_weight`, `strip_punctuation`, `split_into_sentences`, `group_lines`
`src/data_prep.py`	Stream Wikipedia, parse articles, build windows and pairs, split by article	never uses a word-character regex; shuffles sentences inside windows
`src/punctuation/dataset.py`	Align word labels to subwords	first subword labelled, rest `-100`
`src/punctuation/train.py`	Train the token classifier	LoRA, class-weighted loss, best boundary F1
`src/punctuation/predict.py`	Predict one label per word	overlapping windows + triangular weights
`src/ordering/pairs.py`	Build balanced pairs	positive `(A,B,1)` and negative `(B,A,0)`
`src/ordering/train.py`	Train the pair classifier	`[CLS] A [SEP] B [SEP]`, best accuracy
`src/ordering/decode.py`	Matrix -> order	Borda, clusters, beam, TF-IDF ablation, tau
`src/translation/indictrans.py`	Hindi -> English	IndicProcessor, tags, batch generation
`src/pipeline.py`	Run all stages	load/free one model at a time; write four files per sample
`src/evaluate.py`	Measure all stages	punctuation, ordering, translation; JSON report

15.2 The `baseline/` folder

The baseline is the first, rule-based attempt:

File	What it does
`hindi_utils.py`	Unicode-safe `str.split` tokenizer that removes `NULL`
`segment.py`	Dynamic-programming sentence splitter using verb endings, starters, `NULL` hints, and a length prior
`order.py`	TF-IDF (char 2-4 grams) + KMeans + greedy ordering, with a synthetic Kendall's tau check
`translate.py`	`Helsinki-NLP/opus-mt-hi-en` MarianMT wrapper
`pipeline.py`	End-to-end rule baseline

It is kept for comparison and to show the evolution of the work. The final pipeline replaces every heuristic with a trained model. The baseline is **not** used by `src/`.

15.3 The `lessons/` folder

These are early, generic NLP exercises, not part of the final assignment:

- `lesson_01_basics.ipynb`: environment smoke test (Python, NumPy, pandas).

- `lesson_02_tokenization.ipynb`: NLTK word and sentence tokenization on English.
- `lesson_03_preprocessing.ipynb`: cleaning, stopword removal, stemming, lemmatization.
- `lessons/preprocessing.py`: `clean_text`, `tokenize`, `stem_tokens`, `lemmatize_tokens`, `sentence_split`.
- `lessons/train.py`: TF-IDF + logistic regression on `data/raw/dataset.csv` (a generic text-classification exercise).

They document the learning path: before building transformers, the project established the classical NLP basics.

16. Environment and dependency saga

> **APPROACH CHOSEN.** Pin an exact, verified combination of libraries rather than chasing "latest". Write the compatibility reasons into the requirements file itself. Install IndicTransToolkit separately because it may need a source build. This turns a fragile stack into a reproducible one.

The final verified combination is pinned in `requirements-colab.txt`:

```
transformers==4.44.2
tokenizers==0.19.1
datasets==3.6.0
huggingface-hub>=0.24,<1.0
accelerate>=0.34,<2.0
peft==0.13.2
sentencepiece>=0.1.99
sacremoses
sacrebleu
protobuf
scikit-learn
scipy
numpy
pandas
tqdm
```

plus `IndicTransToolkit==1.1.1`, installed by `scripts/colab_setup.sh` because it may need a source build.

16.1 Why each pin exists

Pin	Reason
`transformers==4.44.2`	transformers 5 removes modules that IndicTrans2's remote code and IndicTransToolkit import. 4.44.x is the verified compatible line.
`tokenizers==0.19.1`	version required by transformers 4.44.x; mismatches cause fast-tokenizer errors.
`datasets==3.6.0`	datasets 4/5 require `huggingface-hub>=1.31`, which transformers 4.44 does not support.

Pin	Reason
`huggingface-hub>=0.24,<1.0`	keeps the hub client compatible with transformers 4.44 and datasets 3.6.
`accelerate>=0.34,<2.0`	Trainer integration; newer majors can change APIs.
`peft==0.13.2`	newer peft needs `torch.distributed.tensor`, which is absent from some macOS torch builds.
`sentencepiece`	tokenizer backend for IndicTrans2.
`sacremoses`	text normalization used by some tokenizers.
`sacrebleu`	BLEU/chrF evaluation.

> **The transformers 5 problem in one sentence.** The model code was written against transformers 4.x module paths; transformers 5 reorganized those paths, so importing the model fails even though the model weights are fine. Pinning 4.44.2 is the pragmatic fix.

16.2 The MPS attention problem

On Apple Silicon, PyTorch's MPS backend does not implement dropout inside `scaled_dot_product_attention` in this torch build. Transformers 4.x uses SDPA by default, so training fails or produces incorrect results. The project detects MPS and switches to **eager attention**:

```
def attention_implementation():
    import torch
    if torch.cuda.is_available():
        return None
    mps = getattr(torch.backends, 'mps', None)
    if mps is not None and mps.is_available():
        return 'eager'
    return None
```

Both training scripts pass `attn_implementation='eager'` when needed. On CUDA, the faster default is kept.

16.3 The gated IndicTrans2 problem

The official `ai4bharat/indictrans2-indic-en-dist-200M` is gated. Without a Hugging Face token, loading returns 401. For local verification, the project uses a merged, ungated mirror at `models/translation/indictrans2-indic-en-200M/`. The wrapper accepts either a Hub id or a local path, so no code change is needed. On Colab, authenticate and accept the license, then use the official id. The code never silently falls back to a different translation model; if the model cannot load, it raises an error.

16.4 The data-preparation thread hang

Streaming Wikipedia with `datasets` can leave non-daemon threads alive after the script finishes, so the process may appear to hang at exit. The data is already written. In practice, run data preparation in the background and poll the log, or interrupt once the summary is written. This is a tooling quirk, not a logic bug.

16.5 Two local environments

The repository has two local environments:

- `.venv/` (Python 3.13): CPU libraries, Jupyter, scikit-learn. Used for `setup_check.py` and CPU-only CLIs.
- `.venv.backup-20261109-124948/` (Python 3.12): torch 2.15, transformers 4.44.2, peft 0.13.2, datasets 3.6.0. Used for the full deep-learning pipeline on MPS.

The Jupyter kernel `nlp-zero-to-hero-mps` points at the second environment. The `nlp-zero-to-hero` kernel points at the first. Use the MPS kernel for the demo notebook.

16.6 Setup commands

Fresh local setup:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install --upgrade -r requirements-colab.txt
python -m pip install 'IndicTransToolkit==1.1.1'
```

If the toolkit wheel fails, build from source as `scripts/colab_setup.sh` does:

```
git clone --depth 1 https://github.com/VarunGumma/IndicTransToolkit.git /tmp/IndicTransToolkit
python -m pip install --editable /tmp/IndicTransToolkit --no-build-isolation
```

> **Check yourself.** Why not just install the latest versions of everything? *Because these libraries are coupled: transformers constrains tokenizers, datasets constrains huggingface-hub, and peft constrains torch. The "latest" combination is frequently incompatible, especially across major versions. Pinning is how you make a pipeline reproducible.*

17. Data preparation deep dive

> **APPROACH CHOSEN.** Create supervision by hiding information that Wikipedia already contains. Punctuation is removed and stored as word labels; sentence order is shuffled inside windows; ordering pairs come from consecutive sentences. Split by article, cap examples for tractability, and never synthesize labels with an LLM.

`src/data_prep.py` turns raw Wikipedia into two datasets. It is worth understanding line by line because the model can only be as good as its labels.

17.1 Streaming Wikipedia

```
stream = load_dataset(WIKI_DATASET, WIKI_CONFIG, split='train', streaming=True)
```

- `wikimedia/wikipedia` with config `20231101.hi` is the November 2023 Hindi Wikipedia dump.

- `streaming=True` means rows are downloaded on demand; the full dump is never stored.
- `scan_multiplier=3` means the loader scans up to 9,000 pages to keep 3,000 usable articles. Some pages are lists, stubs, or otherwise unusable.

17.2 Parsing an article

```
ARTICLE_PATTERN = re.compile(r'(?<=[।॥?!])\s+')
```

Steps in `parse_article`:

1. Remove citation markers like `[12]` with a regex.
2. Collapse whitespace.
3. Split after sentence-final marks using the lookbehind regex.
4. For each token, call `strip_punctuation()` to separate the word from its trailing mark.
5. Keep tokens that contain Devanagari or a digit. This drops stray Latin words, URLs, and markup.
6. Accumulate words until a boundary label. If the sentence length is between 5 and 40 words, keep it. Otherwise discard it.
7. If the article yields fewer than 3 sentences, discard the article.

> **Why discard short sentences?** Wikipedia has many fragments (headings, list items, captions). Training on them would teach the model that a sentence can be two words long, which is not true for the assignment's prose. The 5-word minimum is a data-cleaning choice, and it also explains why dialogue-like samples are over-segmented at inference.

17.3 Punctuation windows

`make_punctuation_windows` slides over the article's sentences:

- Start at sentence `index`.
- Add sentences until the total would exceed 96 words or reaches at least 64 words.
- Keep the window if it has 32-96 words and at least 2 sentences.
- **Shuffle the sentences inside the window** with the seeded RNG.
- Concatenate their words and labels.
- Advance by at least 40 words (stride), moving whole sentences.

Why shuffle? The assignment input has shuffled sentences. If training windows kept the original order, the model could learn cross-sentence continuity cues that will not exist at inference. Shuffling forces the model to rely on local word context, which is the right signal for punctuation.

The window is measured in **words**, not subwords. A 64-word window may become 100+ subword tokens; `MAX_SEQ_LENGTH=128` truncates the tail. The first-subword label alignment ensures the remaining words still have labels.

17.4 Splitting by article

Article IDs are shuffled with a fixed seed, then split 90/5/5. The same split is used for punctuation and ordering. This prevents leakage and makes the two stages comparable.

17.5 Capping examples

Training punctuation is capped at 25,000 windows, ordering at 40,000 pairs. Evaluation is capped at 3,000 pairs (and the punctuation test is used in full unless `--max-punct-examples` is set). The cap keeps training time bounded and makes runs comparable.

> **Subtle detail.** The punctuation loop breaks out of the article loop once it has 1.2x the window limit, then shuffles and caps. This means the training set is a random sample of the first ~30,000 windows, not a random sample of all possible windows. For a course project this is fine; for a production system you would stream all articles and sample uniformly.

17.6 Ordering pairs

`src/ordering/pairs.py:13-49` builds pairs from consecutive sentences:

- Choose up to 30 positions per article, evenly spaced if the article is longer.
- For each position, take `(sentences[i], sentences[i+1])`.
- Emit a positive example `(A, B, label=1)`.
- Emit a negative example `(B, A, label=0)`.

This gives a balanced binary dataset and teaches the model the direction of local narrative flow. It does not teach global order directly; the decoder must combine local preferences into a global permutation.

17.7 The held-out ordering articles

For Kendall's tau, the project saves 40 test articles with 8-70 sentences in their original order:

```
{'article_id': 'hi-01234', 'sentences': ['...', '...', '...']}
```

At evaluation time, each article is shuffled with a fixed seed, the model scores all pairs, each decoder recovers an order, and tau is computed against the original indices. This measures full-order recovery, not just pair accuracy.

17.8 Output schemas

Punctuation JSONL row:

```
{
  'id': 'hi-00001-0',
  'article_id': 'hi-00001',
  'split': 'train',
  'words': ['भारत', 'एक', 'विशाल', 'देश', 'है'],
  'labels': ['none', 'none', 'none', 'none', 'sentence_end']
}
```

Ordering JSONL row:

```
{
  'id': 'hi-00001-10',
  'sentence_a': 'पहला वाक्य',
  'sentence_b': 'दूसरा वाक्य',
  'label': 1
}
```

Ordering evaluation row:

```
{'article_id': 'hi-01234', 'sentences': ['पहला', 'दूसरा', 'तीसरा']}
```

17.9 Reproducing data preparation

```
python -m src.data_prep --num-articles 3000 --max-train-pairs 40000
```

Smaller first run:

```
python -m src.data_prep --num-articles 1200 --max-train-pairs 20000
```

The command writes `data/processed/punctuation/{train,val,test}.jsonl`, `data/processed/ordering/{train,val,test}.jsonl`, `data/processed/ordering/ordering_eval.jsonl`, and two copies of `data_prep_summary.json` (in `data/processed/` and `reports/`).

> **Check yourself.** Why is the same article split used for both stages? *Because the two stages are trained and evaluated on the same documents, making the evaluation comparable and preventing leakage across stages. It also simplifies the story: the test set is a set of unseen articles, period.*

18. Stage 1: Punctuation restoration

> **APPROACH CHOSEN.** Treat punctuation as word-level token classification. Label each word with one of five marks, align the label to the first subword, use a class-weighted loss, and reconstruct sentences from boundary labels. This is the simplest formulation that preserves the input words and fits the memory budget.

18.1 Problem formulation

Input: a list of Hindi words `[w_1, ..., w_n]` with no punctuation. Output: a label `y_i` for each word, where `y_i` is one of:

Label	Mark	Meaning
`none`	`''`	no punctuation after this word
`comma`	`,`	comma after this word
`sentence_end`	`।`	sentence ends here
`question`	`?`	question ends here
`exclamation`	`!`	exclamation ends here

The label belongs to the word *before* the mark. For example, in `हुआ।`, the word is `हुआ` and the label is `sentence_end`.

18.2 Why token classification

Alternatives:

- **Sequence-to-sequence rewriting:** generate the punctuated text. More flexible, but can change words, is slower, and is harder to evaluate at the word level.
- **Boundary-only binary classification:** predict boundary vs non-boundary, then a second model chooses the mark. Simpler, but loses the joint information and needs two models.
- **Token classification:** one label per word, words preserved, single model. This is the chosen approach.

18.3 Subword alignment

The dataset code (`src/punctuation/dataset.py`) does:

```
encoding = tokenizer(row['words'], is_split_into_words=True,
                     truncation=True, max_length=128)
word_ids = encoding.word_ids()
labels = [-100] * len(encoding['input_ids'])
previous_word = None
for position, word_id in enumerate(word_ids):
    if word_id is None or word_id == previous_word:
        continue
    previous_word = word_id
    labels[position] = LABEL2ID[row['labels'][word_id]]
```

For `अयोध्या` (pieces `अ`, `##य`, `##ोध`, `##्या`), the label goes on `अ`; the three continuation pieces get `-100`. At inference, the code reads the prediction at the first subword of each word.

18.4 Model and head

`AutoModelForTokenClassification.from_pretrained(BASE_MODEL, num_labels=5, ...)` loads mBERT and adds a linear head. The head maps each 768-dimensional hidden state to 5 logits. The LoRA adapters are attached to the query and value projections.

18.5 Class-weighted loss

`WeightedTrainer` overrides `compute_loss`:

```
loss_function = torch.nn.CrossEntropyLoss(
    weight=self.class_weight_tensor.to(logits.device),
    ignore_index=-100,
)
loss = loss_function(logits.view(-1, logits.size(-1)), labels.view(-1))
```

This is the only non-standard training component. It is necessary because question and exclamation marks are almost absent.

18.6 Inference: sliding windows and triangular weights

`PunctuationRestorer.predict_labels`:

1. Compute windows of 64 words with stride 40.
2. Tokenize each window with `is_split_into_words=True`, `max_length=160`.
3. Run the model in batches of 16.
4. Softmax the logits.
5. For each word, take the **first** subword's probability vector.
6. Weight it by `window_weight(word_id, window_length)`.
7. Accumulate and normalize.
8. Argmax per word.

The triangular weight is the key smoothing trick. Without it, a word at the edge of one window might get a bad prediction that overrides a good prediction from another window.

18.7 Rebuilding sentences

`split_into_sentences(words, labels, with_punctuation=True)` walks the words:

- append `word + PUNCTUATION[label]`,
- if the label is a boundary, close the current sentence.

`with_punctuation=False` gives plain sentences for the ordering model, which was trained on plain text (no punctuation). This is important: the ordering model should see the same distribution it was trained on.

18.8 What the model learns

The model learns local cues:

- a danda or period after a finite verb,
- a comma after a clause or list item,
- sentence endings after particles like `है`, `था`, `हैं`, `गया`,
- question marks after question words or interrogative particles (rarely, because of data scarcity).

It does not learn global discourse structure; that is the ordering stage's job.

18.9 Worked example from the demo notebook

The demo notebook takes the first test window and compares predictions with gold labels:

```
words in window: 73
अपने                 true=none          pred=none
तरह                  true=none          pred=none
का                   true=none          pred=none
राज्य–प्रबन्धन      true=none          pred=none
विषयक                true=none          pred=none
यह                   true=none          pred=none
प्राचीनतम            true=none          pred=none
ग्रन्थ               true=none          pred=none
है                   true=sentence_end pred=sentence_end
इसमें                true=none          pred=none
राज्यव्यवस्था       true=comma         pred=comma
कृषि                 true=comma         pred=comma
window token accuracy: 0.849
```

This window is handled well: the sentence ending after `है` and the commas after `राज्यव्यवस्था` and `कृषि` are correct. The window accuracy (84.9%) is close to the overall token accuracy (87.2%). Note that `राज्य–प्रबन्धन` is a single word containing a hyphen; the tokenizer splits it into pieces, but the label is still applied to the first piece.

18.10 Known limitations

- **Comma F1 is low (0.3161).** Commas are context-dependent and less frequent than sentence endings. The model often misses them.
- **Question/exclamation F1 is 0.** There are only 236 and 153 training examples, and the cap on class weights (20) limits their influence. The model essentially never predicts them.
- **Over-segmentation on dialogue.** Short utterances look like sentence endings. Samples 6 and 7 show this.
- **Truncation.** A 64-word window can exceed 128 subwords; the tail is truncated. This affects long words and complex compounds.
- **Danda vs period.** The model predicts `sentence_end`, which is rendered as `।` regardless of whether the original used `.`. For Hindi output this is appropriate.

> **Check yourself.** Why is the comma the hardest class? *Because comma placement depends on clause structure and style, not just local word identity. A comma can be optional in many positions, so the label is inherently ambiguous. Sentence endings have stronger cues (finite verbs, particles).*

19. Stage 2: Sentence ordering

> **APPROACH CHOSEN.** Learn only pairwise direction, then decode globally. A binary mBERT classifier answers "does A come before B?" for every pair; Borda aggregation turns the probability matrix into a full order. We compare Borda against beam and clustering decoders, and against a TF-IDF-only ablation, rather than assuming one is best.

19.1 Problem formulation

Input: a list of sentences in random order. Output: a permutation that recovers the original narrative order.

The project decomposes this into pairwise decisions: for sentences A and B, predict whether A comes before B. Then a decoder aggregates the pairwise probabilities into a global order.

19.2 Why pairwise

Alternatives:

- **Direct permutation prediction:** a model outputs a permutation of `n` items. Hard to train, variable output size, and expensive.
- **Pointer networks:** generate the order one sentence at a time, attending to the remaining sentences. Elegant, but more complex and harder to fit in the budget.
- **Pairwise ranking:** train a binary classifier on pairs, then decode. Simple, generalizes to any `n`, and the pairwise model can be reused by multiple decoders.

The pairwise approach is the standard compromise and matches the course notes.

19.3 Building balanced pairs

For consecutive sentences in a Wikipedia article:

```
(A, B, 1)   # A really comes before B
(B, A, 0)   # the reverse is wrong
```

This gives a 50/50 label distribution, so accuracy is meaningful and no class weighting is needed. The model sees both directions of the same pair, which also teaches symmetry.

19.4 Model

`AutoModelForSequenceClassification.from_pretrained(BASE_MODEL, num_labels=2)` loads mBERT and adds a linear head on the `[CLS]` vector. The input format is:

```
[CLS] sentence A [SEP] sentence B [SEP]
```

The tokenizer creates **token type IDs** (segment IDs): 0 for A, 1 for B. BERT's embeddings add a segment embedding, so the model knows which sentence is which. The `[CLS]` representation is passed to the classifier, which outputs two logits.

19.5 Inference: the score matrix

`OrderScorer.score_matrix(sentences)`:

1. Enumerate all `i < j` pairs.
2. Batch them through the model.
3. For each pair, take `softmax(logits)[1] = P(A before B)`.
4. Fill `M[i,j] = p`, `M[j,i] = 1 − p`.

5. Diagonal stays 0.5.

The matrix is symmetric-complement, so every decoder can look up either direction.

19.6 Decoder 1: Borda

For each sentence `i`:

```
s_i = (1 / (n - 1)) * sum_{j != i} M[i, j]
```

Sort sentences by `s_i` descending. A sentence that tends to come before others gets a high score and moves to the front.

Why it works: averaging over all pairs reduces the impact of individual pair errors. It is a Borda/Kemeny-style approximation. The true Kemeny order minimizes the number of pairwise disagreements and is NP-hard; Borda is a cheap approximation.

19.7 Decoder 2: clusters

1. TF-IDF with `analyzer='char_wb'`, `ngram_range=(2,4)`, `sublinear_tf=True`.
2. KMeans with `k = round(n/6)`, clamped to 1-12.
3. For each cluster, choose the medoid: the member with the highest cosine similarity to the cluster centroid.
4. Order the medoids with a greedy pass over the medoid submatrix.
5. Beam-search inside each cluster.
6. Concatenate cluster orders.

Why cluster? Narrative order often moves between topics. Grouping topically similar sentences reduces the number of expensive pair comparisons and can improve local coherence. In the evaluation, clusters underperform Borda on average, but it is a useful comparison and follows the notes closely.

19.8 Decoder 3: beam

Beam search over chains:

- Start from the `beam_width` sentences with the highest earliness score.
- Expand each partial chain by appending a remaining sentence.
- Score the chain by the sum of adjacent pair probabilities.
- Keep the top `beam_width` chains.
- Return the best full chain.

Beam is greedy in the sense that it only scores adjacent transitions, so an early mistake can propagate. It performs worst of the three on average.

19.9 TF-IDF ablation

A pure lexical baseline:

- TF-IDF char 2-4 grams,
- cosine similarity matrix,

- diagonal zeroed,
- greedy order by similarity to the current sentence.

This measures how much the mBERT pair model adds over simple lexical cohesion. The answer: a lot, but not enormous. Borda tau 0.2007 vs TF-IDF 0.0631.

19.10 Kendall's tau evaluation

For each of 40 held-out articles:

1. Start from the true order `[0, 1, ..., n-1]`.
2. Shuffle with a fixed seed to get `shuffled_indices`.
3. Score the shuffled sentences.
4. Decode to positions in the shuffled list.
5. Map back to original indices.
6. Compute `kendalltau(recovered, truth)`.

One matrix is computed per article and reused by all decoders, so comparing them is cheap.

19.11 Worked example from the demo notebook

The demo takes the first test pair:

```
gold label: 1 (1 = A before B)
P(A before B): 0.711
P(B before A): 0.289
A: यह बाद में ऑगस्ट कॉम्ट(1798–1857 द्वारा 1838 में स्थापित किया गया
B: इससे पहले कॉम्ट ने सामाजिक भौतिकी शब्द का इस्तेमाल किया था लेकिन बाद में वह दूसरों द्वारा अपनाया गया विशेष रूप से बेल्जि
```

The model assigns 0.711 to the correct direction, a clear but not overwhelming signal. The text is noisy: the first sentence is truncated mid-parenthesis, and the second is a long clause. This is typical of real Wikipedia text and explains why ordering is hard.

19.12 Known limitations

- **Weak global tau (0.2007).** Pairwise accuracy 65% does not translate into strong full-order recovery.
- **Topical vs narrative order.** The model learns local coherence, but Wikipedia articles often move between topics in ways that are not predictable from adjacent sentences.
- **Long articles.** Quadratic pair count and truncation at 192 tokens. For 99 sentences, the model makes 4,851 calls.
- **No global constraints.** Borda can produce an order that violates transitivity (A before B, B before C, but C before A in the scores). The decoder does not enforce a consistent global ranking.
- **Training data size.** The local adapter saw only 4,000 pairs. The full recipe uses 40,000.

> **Check yourself.** Why can a model with 65% pair accuracy produce an order with tau close to zero?
> *Because errors are correlated and early mistakes propagate. Global order requires getting many pairs right simultaneously, and a few confidently wrong pairs near topic boundaries can scramble large blocks.*

20. Stage 3: Hindi-to-English translation

> **APPROACH CHOSEN.** Use a dedicated 200M Indic-to-English encoder-decoder, not a generic multilingual model and not an LLM. Translate sentence by sentence to preserve exact alignment with the Hindi sentences and the groups of five. Use IndicProcessor for language tags and preprocessing.

20.1 Model

`ai4bharat/indictrans2-indic-en-dist-200M` is a 200M-parameter encoder-decoder transformer for Indic-to-English translation. Its configuration (from `models/translation/indictrans2-indic-en-200M/config.json`):

Property	Value
Architecture	`IndicTransForConditionalGeneration`
Encoder layers	18
Decoder layers	18
Model dimension	512
Attention heads	8
Feed-forward size	2,048
Encoder vocabulary	122,706
Decoder vocabulary	32,296
Max positions	256
Dropout	0.2
Tied embeddings	yes

The model uses a custom `IndicTrans` architecture with `trust_remote_code=True`.

20.2 IndicProcessor

IndicTrans2 expects language tags and preprocessing. `IndicProcessor(inference=True)`:

- adds the source/target tags (`hin_Deva`, `eng_Latn`),
- normalizes and tokenizes text according to the model's conventions,
- postprocesses generated text.

The wrapper calls:

```
prepared = self.processor.preprocess_batch(batch, src_lang='hin_Deva',
                                           tgt_lang='eng_Latn')
```

Then tokenizes and generates.

> **Subtle bug fixed during development.** The tokenizer is called *without* `src_lang` / `tgt_lang` kwargs because the processor already inserted the tags. Passing them again causes errors. The wrapper also sets `return_attention_mask=True` and `clean_up_tokenization_spaces=True`, and supplies `config.vocab_size` from the decoder vocabulary when the config lacks a top-level `vocab_size`.

20.3 Generation

```
generated = self.model.generate(
    **encoded,
    num_beams=4,
    num_return_sequences=1,
    max_length=256,
    early_stopping=True,
)
```

- **Beam search (4 beams):** explores four partial translations, which usually beats greedy decoding.
- **Max length 256:** enough for the assignment's sentences; longer inputs are truncated.
- **Early stopping:** stop when all beams are finished.

The output is decoded with `skip_special_tokens=True`, then postprocessed by IndicProcessor.

20.4 Batching and memory

- Batch size defaults to 4 in the pipeline and can be raised with `--translation-batch-size`.
- On CUDA, dtype is fp16; on MPS/CPU, fp32.
- The model is loaded with `low_cpu_mem_usage=True`.
- After translation, `close()` deletes the model and calls `free_memory()`.

20.5 Translation quality

The evaluation on 100 IITB sentences gives BLEU 34.125 and chrF 59.686. Example outputs (from `reports/evaluation.json`) show:

Hindi	Reference	Hypothesis
आपकी कार में ब्लैक बॉक्स?	A black box in your car?	A black box in your car?
आम तौर पर हाईवे नियोजन जैसा उबाऊ काम भी अचानक गहन बहस तथा जीवंत गठबंधनों का मुद्दा बन गया है।	The usually dull arena of highway planning has suddenly spawned intense debate and colorful alliances.	Even boring work such as highway planning in general has suddenly become an issue of intense debate and lively alliances.

The first is exact; the second is a paraphrase with different wording but preserved meaning. This is typical of BLEU: exact matches are rewarded heavily, paraphrases less so.

20.6 Why not a larger model or an LLM

- The assignment forbids LLMs.
- A 200M model fits the 1.5 GB budget; a 7B model does not.

- IndicTrans2 is specialized for Indic languages and outperforms generic small models on this task.
- A dedicated translation model is easier to evaluate and reproduce than a prompted LLM.

20.7 Known limitations

- **Gated access.** The official model requires authentication; the local mirror is a workaround.
- **Sentence-level translation.** Each sentence is translated independently, so cross-sentence discourse is lost.
- **Truncation.** Sentences longer than 256 tokens are cut.
- **Paraphrase.** BLEU penalizes valid paraphrases; chrF is more forgiving.
- **No glossary control.** Domain terms may be translated inconsistently.
- **Local mirror provenance.** The local merged mirror should be replaced with the official checkpoint for a final submission if possible.

> **Check yourself.** Why translate sentence by sentence instead of translating the whole document at once? *Because the assignment requires sentence-aligned Hindi/English outputs and groups of five. Sentence-level translation keeps alignment exact. Document-level translation might improve coherence but would require alignment heuristics and could exceed the model's context.*

21. Pipeline orchestration

> **APPROACH CHOSEN.** Orchestrate with a thin, explicit script rather than a framework. Load all text first, then run stage 1 on every file, free it, run stage 2 on every file, free it, run stage 3 on all sentences, and write aligned outputs. The code is short enough to audit and makes the memory story obvious.

`src/pipeline.py` is the glue. It is short, and every line matters.

21.1 Reading tokens

```
def read_tokens(path):
    text = Path(path).read_text(encoding='utf-8')
    raw_tokens = text.split()
    words = [token for token in raw_tokens if token != 'NULL']
    return words, len(raw_tokens) - len(words)
```

- `str.split()` handles arbitrary whitespace and preserves Devanagari.
- `NULL` tokens are removed. The count is recorded for the report.
- All records are loaded before any model runs, so the memory used by raw text is negligible.

21.2 Stage 1

```
restorer = PunctuationRestorer(args.punctuation_model, memory_gb=args.memory_gb)
for stem, record in records.items():
    labels, sentences, plain_sentences = restorer.restore(record['words'])
    ...
restorer.close()
```

The restorer returns three things:

- `labels`: one label ID per word,
- `sentences`: punctuated Hindi sentences,
- `plain_sentences`: the same sentences without punctuation, for the ordering model.

21.3 Stage 2

```
scorer = OrderScorer(args.ordering_model, memory_gb=args.memory_gb)
for stem, record in records.items():
    order = order_sentences(record['sentences_plain'], scorer,
                            strategy=args.strategy, ...)
    record['ordered_hi'] = [record['sentences_hi'][i] for i in order]
    record['groups_hi'] = group_lines(record['ordered_hi'], GROUP_SIZE)
scorer.close()
```

Note the index mapping: the order is computed on **plain** sentences, but applied to the **punctuated** sentences. The two lists have the same length and the same sentence boundaries, so the mapping is valid. This is a small but important detail: the ordering model sees the text distribution it was trained on, while the output keeps punctuation.

21.4 Stage 3

```
flat_sentences = [s for record in records.values() for s in record['ordered_hi']]
translations = translator.translate(flat_sentences,
                                    batch_size=args.translation_batch_size)
```

All sentences from all files are flattened into one list and translated in batches. Then the translations are sliced back per file using the sentence counts. This maximizes batch utilization.

Grouping:

```
record['groups_en'] = [
    ' '.join(group)
    for group in group_lines(record['ordered_en'], GROUP_SIZE)
]
```

The Hindi groups are built from sentences, the English groups from translations, and both use the same `GROUP_SIZE=5`, so they stay aligned.

21.5 Saving

For each file, four outputs are written. The summary records words, `NULL` removed, sentence count, group count, and stage timings. It is saved to `reports/run_summary.json`.

21.6 Failure behavior

- Missing punctuation adapter: `FileNotFoundError` with a message telling you to train it.
- Missing ordering adapter: same.
- Missing IndicTrans2 or toolkit: `ImportError` with a setup hint.
- No input files: `SystemExit` with the directory path.
- Translation failure: raises; there is no fallback model.

This is deliberate. Silent fallbacks make results hard to reproduce and can hide a broken setup.

21.7 CLI options

Option	Default	Meaning
`--input-dir`	`data/raw`	where the `.txt` inputs live
`--output-dir`	`outputs`	where the four files per input are written
`--punctuation-model`	`models/punctuation`	adapter directory
`--ordering-model`	`models/ordering`	adapter directory
`--translation-model`	official Hub id	local path or Hub id
`--limit`	none	process only the first N files
`--strategy`	`borda`	`borda`, `clusters`, `beam`
`--n-clusters`	auto	override cluster count
`--beam-width`	8	beam width for beam/cluster decoding
`--translation-batch-size`	4	batch size for translation
`--memory-gb`	1.5	CUDA memory cap
`--no-translate`	off	skip stage 3

> **Check yourself.** Why load all records before running any model? *Because the models are the memory bottleneck, not the text. Loading the text first lets the pipeline process every file with one model load, instead of reloading the model per file.*

22. Evaluation

> **APPROACH CHOSEN.** Evaluate each stage on its own held-out data with the metric that matches the task, and write a single JSON report. Optional stages skip gracefully when data or models are unavailable. The evaluation uses the same inference code as the pipeline, so the numbers reflect deployment behavior.

`src/evaluate.py` measures each stage independently and writes `reports/evaluation.json`.

22.1 Punctuation evaluation

- Load 1,500 test windows (108,898 words).
- Run the full sliding-window inference (`PunctuationRestorer.predict_labels`).
- Flatten true and predicted labels.
- Compute token accuracy, boundary precision/recall/F1, and per-class F1.

This is the same inference path used by the pipeline, so the score reflects deployment behavior.

22.2 Ordering evaluation

- Load 3,000 balanced test pairs.
- Predict labels with `OrderScorer.predict_pair_labels`.
- Compute accuracy and macro precision/recall/F1.
- Load 40 held-out articles, shuffle each, score all pairs, decode with all strategies plus TF-IDF, and compute Kendall's tau.

22.3 Translation evaluation

- Stream `cfilt/iitb-english-hindi` test split.
- Take the first 100 (or `--max-translation-examples`) Hindi/English pairs.
- Translate the Hindi with the same wrapper.
- Compute corpus BLEU and chrF with `sacrebleu`.
- Save five sample translations in the report.

If the dataset cannot be downloaded or `sacrebleu` is missing, the stage is marked `'skipped'` with a reason instead of failing the whole evaluation.

22.4 Report structure

```
{
  'no_llm': true,
  'memory_limit_gb': 1.5,
  'punctuation': {'status': 'ok', '...': '...'},
  'ordering': {'status': 'ok', '...': '...'},
  'translation': {'status': 'ok', '...': '...'}
}
```

The `no_llm` flag is a deliberate assertion: the evaluation itself uses no LLM.

22.5 Running subsets

You can skip stages or cap examples:

```
python -m src.evaluate --skip-translation
python -m src.evaluate --skip-punctuation --skip-ordering \
    --max-translation-examples 50
python -m src.evaluate --max-punct-examples 300 --max-order-pairs 500 \
    --max-order-articles 10 --report /tmp/quick_eval.json
```

This is useful when iterating: punctuation is fast, ordering is slow, translation is optional.

22.6 Interpreting the results honestly

- The punctuation model is good at sentence endings, weak at commas, blind to question/exclamation.
- The ordering model is better than chance and better than TF-IDF, but full-order recovery is weak.
- The translation model is solid on a standard benchmark but varies by sentence.
- The pipeline outputs are usable, especially for prose; dialogue-like inputs are over-segmented.

> **Check yourself.** Why evaluate ordering with both pair accuracy and Kendall's tau? *Pair accuracy measures the classifier in isolation; tau measures the end-to-end ordering quality after decoding. A model can be decent at pairs but poor at full order, so both are needed.*

23. Reproducibility: from zero to outputs

> **APPROACH CHOSEN.** Provide two reproducible recipes: the full Colab/T4 run and the verified local subset run. Document exactly what was executed, save all reports, and publish an executed notebook. Never present a documented recipe as if it had been run.

23.1 Full Colab recipe

```
# 1. Environment
bash scripts/colab_setup.sh

# 2. Data
python -m src.data_prep --num-articles 3000 --max-train-pairs 40000

# 3. Train
python -m src.punctuation.train --epochs 3 --batch-size 8 --learning-rate 3e-4
python -m src.ordering.train    --epochs 2 --batch-size 8 --learning-rate 2e-4

# 4. Evaluate
python -m src.evaluate --max-translation-examples 200

# 5. Run
python -m src.pipeline --strategy borda
```

On a T4, this is the intended full run. The notebook automates the same sequence. The official IndicTrans2 checkpoint requires accepting the license and logging in.

23.2 Local verified recipe

```
python -m src.data_prep --num-articles 3000 --max-train-pairs 40000
python -m src.punctuation.train --epochs 2 --batch-size 16 \
    --learning-rate 3e-4 --max-train-samples 4000
python -m src.ordering.train --epochs 1 --batch-size 16 \
    --learning-rate 2e-4 --max-train-samples 4000
python -m src.evaluate \
    --translation-model models/translation/indictrans2-indic-en-200M \
    --max-translation-examples 100
python -m src.pipeline \
    --translation-model models/translation/indictrans2-indic-en-200M \
    --translation-batch-size 8
```

This produced the adapters, the seven output sets, and the three reports used throughout this guide.

23.3 Chronology of the build

Phase	What was done	Outcome
1	Built the generic NLP lessons and environment check	classical tokenization/preprocessing foundation
2	Wrote a rule-based baseline (DP segmentation, TF-IDF ordering, OPUS-MT)	working but brittle; kept for comparison
3	Chose mBERT + LoRA for stages 1-2 and IndicTrans2 for stage 3	fits the no-LLM and 1.5 GB constraints
4	Built the Wikipedia data pipeline	3,000 articles; 25k windows; 40k pairs; held-out articles
5	Fixed dependency incompatibilities (transformers 5, peft/torch, datasets/hub)	pinned verified versions
6	Fixed MPS attention and IndicTrans2 tokenizer/config issues	local MPS run works
7	Trained subset adapters (4k/4k)	punctuation boundary F1 0.8633; ordering macro F1 0.6534
8	Ran the full pipeline on all seven samples	443 sentences; 1-2 minute stages on MPS
9	Evaluated all stages	boundary F1 0.8842; Borda tau 0.2007; BLEU 34.125
10	Built and executed the demo notebook	visible outputs, all seven files aligned
11	Published the repository and notebook to GitHub	public repo
12	Attempted full-data local training	stopped; subset adapters remain; full recipe documented

23.4 How to verify the results

```
# Compile and import
python -m compileall -q src
python -c 'import src.pipeline, src.evaluate, src.punctuation.train, src.ordering.train'

# Check training summaries
python - <<'PY'
import json
for path in ['models/punctuation/training_summary.json',
             'models/ordering/training_summary.json']:
    data = json.load(open(path))
    print(path, data['trainable_parameters'], data['total_parameters'])
    print('  test:', {k: v for k, v in data['test_metrics'].items()
                      if 'f1' in k or 'accuracy' in k})
PY

# Re-run individual evaluations
python -m src.evaluate --skip-ordering --skip-translation --report /tmp/punct.json
python -m src.evaluate --skip-punctuation --skip-translation --report /tmp/order.json
python -m src.evaluate --skip-punctuation --skip-ordering --report /tmp/trans.json

# End-to-end smoke test on one file
python -m src.pipeline --limit 1 --output-dir /tmp/pipeline-smoke
```

23.5 What is not done

- Full-data training was not completed locally. The adapters are subset adapters.
- The notebook has not been run top to bottom on a Colab T4 in this repository. The local demo notebook has been executed.
- There is no automated test suite (no pytest). Verification is via scripts and reports.
- The local IndicTrans2 mirror should be replaced with the official gated checkpoint for a final submission if access is available.

> **Check yourself.** Why is “it runs on my machine” not reproducibility? *Because dependencies, hardware kernels, seeds, and data versions can all differ. Reproducibility requires pinned versions, documented commands, saved reports, and a clear statement of what was and was not run.*

Part III — The notebooks, cell by cell

24. `colab_end_to_end.ipynb` — the full recipe

> **APPROACH CHOSEN.** Put the entire full-scale workflow in one notebook: install, prepare data, train both adapters, evaluate, run the pipeline, and save artifacts. The notebook is the executable version of the README recipe, not a separate demo.

This notebook has 28 cells: markdown explanations interleaved with code. It is designed for a Colab T4 GPU.

24.1 Cell map

Cell	Type	Purpose
0	markdown	title, stage table, GPU instruction
1	markdown	project location instructions
2	code	`os.chdir` to project, `sys.path` setup
3	code	torch/CUDA diagnostics
4	markdown	dependency install explanation
5	code	`!bash scripts/colab_setup.sh`
6	markdown	imports, seeds, memory-cap explanation
7	code	imports, `set_seed`, `limit_gpu_memory(1.5)`
8	markdown	data-preparation explanation
9	code	`!python -m src.data_prep --num-articles 3000 --max-train-pairs 40000`
10	code	read and print data summary; show a sample
11	markdown	stage 1 explanation (labels, LoRA, class weights)
12	code	`!python -m src.punctuation.train --epochs 3 --batch-size 8 ...`
13	code	read punctuation training summary
14	markdown	stage 2 explanation (pair format, decoders)
15	code	`!python -m src.ordering.train --epochs 2 --batch-size 8 ...`
16	code	read ordering training summary
17	markdown	evaluation explanation
18	code	`!python -m src.evaluate --max-...`
19	code	print evaluation JSON
20	markdown	pipeline explanation (four files per input, groups of five)
21	code	`!python -m src.pipeline --strategy borda --memory-gb 1.5`
22	code	print the first four Hindi/English groups of sample-1

Cell	Type	Purpose
23	markdown	memory/timing explanation
24	code	read and print `run_summary.json`
25	markdown	save-results explanation
26	code	zip outputs, copy reports
27	markdown	notes, limitations, troubleshooting

24.2 Cell-by-cell notes

Cells 0-3: setup and diagnostics. The notebook sets `PROJECT_DIR = '/content/nlp-zero-to-hero'`, checks that it exists, changes into it, and inserts it into `sys.path`. It then prints the torch version, whether CUDA is available, the GPU name, and total GPU memory. If CUDA is not available, the training cells will be extremely slow; switch the runtime to GPU.

Cell 5: dependencies. It runs `scripts/colab_setup.sh`, which upgrades pip, installs `requirements-colab.txt`, and installs `IndicTransToolkit` (wheel or source). It also prints torch and CUDA information. This cell takes a few minutes on a fresh runtime.

Cell 7: seeds and memory cap. It imports the project constants and utilities, calls `set_seed(42)`, and applies `limit_gpu_memory(1.5)`. On a 15 GB T4, this sets the process memory fraction to `1.5 / 15 = 0.1`. If the process tries to exceed it, CUDA raises an out-of-memory error rather than silently using more.

Cells 8-10: data. Cell 9 builds the datasets. Cell 10 reads `data/processed/data_prep_summary.json`, prints article counts, split sizes, punctuation examples and label counts, ordering pair counts, and shows the first 18 words/labels of a training window. The expected counts are 3,000 articles, 2,700/150/150 splits, 25,000/2,317/2,208 windows, and 40,000/3,000/3,000 pairs.

Cells 11-13: punctuation training. Cell 12 trains for 3 epochs with batch size 8 and LR 3e-4. Cell 13 reads `models/punctuation/training_summary.json` and prints trainable parameters, validation metrics, test metrics, and peak memory. The log also prints the label counts and class weights.

Cells 14-16: ordering training. Cell 15 trains for 2 epochs with batch size 8 and LR 2e-4. Cell 16 prints validation/test metrics and peak memory. The expected pair accuracy is above 0.5; the full-data recipe should improve on the subset result.

Cells 17-19: evaluation. Cell 18 runs `src.evaluate` with caps: 1,200 punctuation examples, 3,000 ordering pairs, 30 articles, 200 translation examples. Cell 19 prints the evaluation JSON. If IndicTrans2 is not authenticated, the translation stage is skipped with a reason; the punctuation and ordering stages still run.

Cells 20-22: pipeline. Cell 21 runs `src.pipeline` with Borda and the 1.5 GB cap. Cell 22 prints the first four Hindi and English groups of sample-1. Each group should contain five sentences, except possibly the last.

Cells 23-24: timing and memory. Cell 24 reads `run_summary.json` and prints the strategy, timings, memory limit, and per-file counts. On a T4, the ordering stage is the slowest; on MPS, the local run took 130.53 seconds for ordering, 102.17 seconds for translation, and 2.31 seconds for punctuation.

Cells 25-26: saving. Cell 26 zips `outputs/` to `/content/hindi_assignment_outputs.zip` and copies `reports/` to `/content/hindi_assignment_reports/`. This is what you attach to a submission.

Cell 27: notes. It restates the no-LLM guarantee, the no-fallback guarantee, the memory strategy, the data limitations, the rarity of question/exclamation marks, the difficulty of ordering, and translation evaluation caveats. It also has troubleshooting bullets for toolkit builds, peft/transformers mismatches, and CUDA OOM.

24.3 How to run it

1. Open the notebook in Colab.
2. `Runtime > Change runtime type > Hardware accelerator: GPU (T4)`.
3. Upload the project folder to `/content/nlp-zero-to-hero` or clone the GitHub repo there.
4. If using the official IndicTrans2, accept the license on Hugging Face and run `huggingface-cli login` (or set `HF_TOKEN`) before cell 21/18.
5. `Runtime > Run all`.
6. Download the zip and reports from `/content/`.

> **Common issue.** The notebook's default `TRANSLATION_MODEL` is the official gated Hub id. If you have not authenticated, translation fails. You can pass a local mirror path in the commands, but the notebook cells as written use the constant. The local demo notebook is the tested path on this machine.

25. `pipeline_demo.ipynb` – the executed local demo

> **APPROACH CHOSEN.** Publish an executed notebook that uses the real adapters and shows visible outputs for every stage. This is the evidence artifact: it proves the local pipeline runs, that Hindi/English outputs align, and that the reported metrics are reproducible on this machine.

This notebook has 13 cells and is already executed. It is the artifact that proves the local pipeline runs with visible outputs.

25.1 Cell map and observed outputs

Cell	Type	What it does	Observed output
0	markdown	title, environment summary	Python 3.12, torch 2.15 MPS, 2-3 minutes
1	code	set project root, print Python version	`Python: 3.12.13`
2	markdown	stage 1 explanation	—
3	code	load punctuation adapter, predict one test window	73 words; token accuracy 0.849; correct `है -> sentence_end`, `राज्यव्यवस्था -> comma`
4	markdown	stage 2 explanation	—
5	code	load ordering adapter, score one test pair	gold 1; P(A before B)=0.711; P(B before A)=0.289
6	markdown	stage 3 explanation	—
7	code	translate three sentences	“India is a huge country.”; “It's a test.”; “Please submit your report by tomorrow.”

Cell	Type	What it does	Observed output
8	markdown	end-to-end artifacts explanation	—
9	code	check alignment of all seven samples	52/11, 55/11, 49/10, 53/11, 55/11, 80/16, 99/20; all aligned
10	markdown	evaluation explanation	subset adapters
11	code	print key metrics	0.8724 / 0.8842; 0.6543; 0.2007; 34.125 / 59.686
12	markdown	reproduce commands	—

25.2 Why the demo matters

- It uses the **actual trained adapters**, not toy weights.
- It shows **visible outputs** for all three stages, including Hindi text.
- It verifies **alignment**: the number of Hindi sentences equals the number of English sentences equals the summary's sentence count, and the same for groups.
- It reports the **subset adapter** scores honestly.

25.3 How to run it

```
source .venv.backup-20261109-124948/bin/activate
jupyter notebook notebooks/pipeline_demo.ipynb
```

Select the `nlp-zero-to-hero-mps` kernel. Run all cells. It takes about 2-3 minutes because it loads both adapters and the local IndicTrans2 model.

> **Note on the warning messages.** Loading the base model with a new classification head prints "Some weights ... were not initialized". This is expected; the adapter supplies the trained head. The warning does not mean the model is broken.

26. The lesson notebooks and legacy baseline

> **APPROACH CHOSEN.** Keep the learning artifacts and the rejected baseline in the repository, clearly labeled. They show the path from classical NLP to the final architecture and make the comparison honest, but they are not imported by the final pipeline.

26.1 Lessons

The three lesson notebooks are generic English NLP exercises:

- **Lesson 01:** imports NumPy and pandas and prints versions. It is an environment smoke test.
- **Lesson 02:** uses NLTK `word_tokenize` and `sent_tokenize` on an English sentence. It demonstrates that tokenization is not trivial: punctuation is separated, contractions may split, sentences are detected.
- **Lesson 03:** uses `lessons/preprocessing.py` to clean text (lowercase, remove URLs, mentions, hashtags, punctuation), tokenize, remove stopwords, stem with Porter, and lemmatize with WordNet. It is a classical preprocessing pipeline, not used by the final Hindi models.

`lessons/train.py` is a TF-IDF + logistic regression text classifier. It expects `data/raw/dataset.csv` with `text` and `label` columns, which is not part of the assignment data. It is an educational example.

26.2 Baseline

The baseline is a complete rule-based pipeline:

- **Segmentation:** dynamic programming over candidate boundaries, scored by verb endings, sentence starters, `NULL` hints, and a length prior.
- **Ordering:** TF-IDF char 2-4 grams + KMeans + greedy medoid ordering.
- **Translation:** `Helsinki-NLP/opus-mt-hi-en` MarianMT.

It works, but it is brittle: the segmentation rules are hand-tuned, the ordering is lexical, and the translation model is generic. The final pipeline replaces every heuristic with a trained model. The baseline remains useful as a comparison and as a record of the engineering path.

> **Check yourself.** Why keep the baseline if it is not used? *Because it documents the alternatives and provides a sanity check. If the trained pipeline were worse than a simple rule-based system, that would be important to know. It also shows the evolution of the solution, which is valuable in a project report.*

Part IV – Results, defenses, and reference

27. Results and limitations

> **APPROACH CHOSEN.** Report every stage separately, state exactly what was trained (subset vs full), and explain what the numbers do and do not mean. Never hide a weak metric behind an aggregate.

27.1 Headline table

Stage	Metric	Value	What it means
Punctuation	token accuracy	0.8724	most words, which are unpunctuated, are classified correctly
Punctuation	boundary F1	0.8842	precision/recall on `sentence_end`, `question`, `exclamation`
Punctuation	comma F1	0.316	commas are hard; many are missed
Punctuation	question/excl F1	0.000	too rare in training (236 and 153 examples)
Ordering	pair accuracy	0.6543	better than chance, better than TF-IDF
Ordering	macro F1	0.6534	balanced across both pair orientations
Ordering	Borda Kendall tau	0.2007	weak-to-moderate full-order recovery
Ordering	clusters tau	0.1223	cluster decoding underperforms
Ordering	beam tau	0.0921	beam variant underperforms with this model
Ordering	TF-IDF tau	0.0631	lexical baseline is much weaker
Translation	BLEU	34.125	acceptable general-purpose output on 100 IITB sentences
Translation	chrF	59.686	character-level overlap is reasonably high

27.2 Interpretation

Punctuation is mostly a sentence-end problem. The model is strong at deciding where a sentence ends. It is weak at commas, because comma placement is stylistic and context-dependent, and the training data has only 3.3% commas. Question and exclamation marks are effectively absent, so no model can learn them from this data. The high token accuracy is partly an artifact of the `none` majority: predicting `none` everywhere would already score about 0.91. Boundary F1 is the honest metric.

Ordering is better than lexical but not solved. Pair accuracy of 0.65 means the model has learned real coherence signals. But full-order recovery via Borda is weak (tau 0.20). Pairwise preferences are noisy and can conflict, and a wrong early decision propagates. Borda is the best of the three decoders; that is why it is the default.

Translation is the strongest stage. BLEU 34 on IITB is a reasonable result for a 200M-parameter Indic→English model on general prose. It is not a claim about this specific noisy, reordered input. Translation is also the least affected by the upstream stages on well-formed sentences, because IndicTrans2 is robust to imperfect punctuation.

27.3 Limitations, stated plainly

1. **Subset adapters.** The shipped adapters were trained on 4,000 punctuation windows and 4,000 ordering pairs, not the full 25,000/40,000. The full recipe is documented, but full-data training was not completed locally.
2. **Synthetic supervision.** Punctuation labels and ordering pairs come from Wikipedia text. The model learns Wikipedia style. Dialogue, poetry, and informal text behave differently.
3. **Rare labels.** Question and exclamation marks are almost absent. Do not expect them.
4. **Ordering ceiling.** With a 64-word window and pairwise BERT scoring, full-document order remains hard. The grouping step hides some errors by keeping groups of five in the same relative order, but it does not fix them.
5. **Translation evaluation.** BLEU/chrF are computed on clean IITB sentence pairs, not on the pipeline's reordered, punctuated output. They measure the translator, not the full pipeline.
6. **Gated model.** The official IndicTrans2 requires license acceptance. A local mirror was used. Results may differ slightly from the official checkpoint.
7. **No test suite.** Verification is by scripts, reports, and the executed notebook, not by unit tests.
8. **Single seed.** All results are from one run with `SEED=42`. Variance is not measured.

27.4 What would improve it most

Change	Expected effect	Cost
Full-data training (25k/40k)	clear improvement in both adapters	GPU hours
Class-balanced sampling for punctuation	better comma recall	retraining
Add question/exclamation data (or augment)	non-zero rare-class F1	data work
Better ordering decoder (e.g., insertion sort with beam)	higher tau than Borda	inference only
Train ordering on longer contexts	better document-level order	memory/time
Evaluate translation on pipeline output with references	true end-to-end quality	annotation effort
Unit tests for alignment and windowing	fewer regressions	engineering time

> **Check yourself.** Why is “boundary F1 0.88” a better headline than “accuracy 0.87”? *Because 91% of words have no punctuation, so accuracy is dominated by the majority class. Boundary F1 measures the decisions that actually matter.*

28. Design decisions, expanded

> **APPROACH CHOSEN.** Make each major decision explicit, with the alternative, the reason, and the cost. This is the material a reviewer or examiner will probe, so it is written as a decision log rather than prose.

D1. Language: Hindi

- **Chosen:** Hindi (Devanagari), with English translation.

- **Alternatives:** a higher-resource language with better tooling.
- **Why:** the task requires Hindi; Devanagari has clear sentence-ending danda, rich morphology, and available multilingual models.
- **Cost:** fewer high-quality tools than English; tokenization is harder.

D2. Tokenization: whitespace, not regex

- **Chosen:** `str.split()`.
- **Alternatives:** `re.findall(r'\w+', text)`.
- **Why:** Devanagari vowel signs are combining marks; `\w+` drops them (`अयोध्या` → `अय`, `ध`, `य`). Whitespace keeps words intact.
- **Cost:** punctuation attached to words must be handled by the models or stripped.

D3. Punctuation: token classification, not seq2seq

- **Chosen:** classify each word into five labels.
- **Alternatives:** generate punctuated text with a seq2seq model; use a large LLM.
- **Why:** one-to-one alignment, fast, small, no hallucination, fits 1.5 GB.
- **Cost:** cannot insert words; rare labels are hard; comma style is limited.

D4. Model: mBERT + LoRA

- **Chosen:** `bert-base-multilingual-cased` with LoRA r=16 on query/value.
- **Alternatives:** full fine-tuning; a larger multilingual model; XLM-R; MuRIL.
- **Why:** mBERT has Hindi coverage, is ~178M params, and LoRA makes training cheap. It was available locally.
- **Cost:** mBERT is older than XLM-R/MuRIL; cased vocab splits Devanagari heavily.

D5. Label alignment: first subword

- **Chosen:** assign the word label to its first subword; ignore the rest.
- **Alternatives:** average subword logits; predict on the last subword; use word-level tokenizer.
- **Why:** Bengali/Hindi word endings carry punctuation cues, but first-subword is simple, stable, and works well in practice.
- **Cost:** loses some information from later subwords; a learned pooling could be better.

D6. Ordering: pairwise binary classification + Borda

- **Chosen:** train `[CLS] A [SEP] B [SEP] → before/after`; decode with Borda.
- **Alternatives:** pointer networks; seq2seq ordering; listwise ranking; topological sort.
- **Why:** pairwise data is easy to generate and the classifier is small and robust. Borda aggregates noisy preferences.
- **Cost:** pairwise preferences can be cyclic; Borda is not optimal; full order is weak.

D7. Ordering training data: balanced both directions

- **Chosen:** for each adjacent pair, add `(A,B,1)` and `(B,A,0)`.
- **Alternatives:** only natural order; sample random pairs.
- **Why:** removes position bias and doubles data.
- **Cost:** model cannot use absolute position; only relative coherence.

D8. Translation: IndicTrans2 200M

- **Chosen:** `ai4bharat/indictrans2-indic-en-dist-200M`.
- **Alternatives:** MarianMT; NLLB; a large LLM; Google Translate API.
- **Why:** Indic-specialized, 200M is feasible, strong Hindi→English quality, runs locally.
- **Cost:** gated license; ~913 MB; 256-token limit; slower on CPU.

D9. Memory: staged load/free vs simultaneous

- **Chosen:** load one model, run its stage on all files, free it, load the next.
- **Alternatives:** load all three and stream.
- **Why:** three models would exceed 1.5 GB; staging caps peak memory at the largest model.
- **Cost:** cannot jointly optimize stages; more I/O.

D10. Evaluation: per-stage, honest subsets

- **Chosen:** held-out windows/pairs/articles and a 100-sentence translation sample.
- **Alternatives:** one end-to-end human evaluation.
- **Why:** fast, reproducible, isolates failures.
- **Cost:** does not measure full-pipeline quality directly.

D11. Notebooks: recipe + executed demo

- **Chosen:** a full Colab recipe notebook and an executed local demo.
- **Alternatives:** only scripts; only a notebook.
- **Why:** the recipe is portable; the demo is evidence.
- **Cost:** two artifacts to maintain; the recipe is not executed here.

D12. No LLM

- **Chosen:** no generative LLM anywhere in the pipeline.
- **Alternatives:** prompt GPT/Gemini for punctuation/ordering/translation.
- **Why:** the assignment forbids an LLM-based solution; also cost, latency, privacy, and determinism.
- **Cost:** lower ceiling on quality, especially ordering.

29. Viva and interview question bank

> **APPROACH CHOSEN.** Prepare short, defensible answers that connect each design choice to a constraint and a measurement. If you can answer the "why not" questions, you understand the project.

29.1 Problem and data

1. **What is the task?** Restore punctuation in unpunctuated Hindi, reorder shuffled sentences into a coherent document, and translate to English.
2. **Why is Hindi hard?** Rich morphology, free word order, and combining marks in Devanagari.
3. **Where does the data come from?** 3,000 Hindi Wikipedia articles, split 2,700/150/150.
4. **How are punctuation labels derived?** From the original punctuation attached to tokens; `none/comma/sentence_end/question/exclamation`.
5. **How are windows built?** 64-word windows, stride 40, kept only if 32-96 words and 5-40-word sentences.
6. **Why stride 40?** It gives overlap so boundary words appear in multiple windows and get averaged.
7. **How are ordering pairs built?** Adjacent sentence pairs, both directions, balanced labels.
8. **Why balanced directions?** To remove position bias and double the data.
9. **Why hold out articles for ordering?** Pairs from the same article would leak context into evaluation.
10. **What is the `NULL` token?** A placeholder in the raw data; it is removed before processing and counted.

29.2 Punctuation

11. **Why token classification?** It preserves alignment and is fast, small, and deterministic.
12. **Why not seq2seq?** It can hallucinate or change words, and alignment is harder.
13. **How do you handle subwords?** Use the first subword's prediction for the word.
14. **Why first subword?** Simple and stable; a learned pooling is future work.
15. **Why class weights?** `none` is ~91%; weights `[0.22, 6.13, 3.47, 20, 20]` counter imbalance.
16. **Why cap weights at 20?** Uncapped `question` would be ~1540, destabilizing training.
17. **How does sliding-window inference work?** Predict per window, accumulate logits per word, average, argmax.
18. **Why `window_weight`?** It counts how many windows scored each word so averaging is correct.
19. **What are boundary labels?** `sentence_end`, `question`, `exclamation`; `none` and `comma` are not boundaries.
20. **Why is comma F1 low?** Commas are stylistic, sparse, and context-dependent.
21. **Why are question/exclamation F1 zero?** Only 236/153 training examples; the model never predicts them.
22. **What is `restore_punctuation`?** A lightweight function that attaches punctuation for display, capped at one terminal mark.
23. **Why not predict punctuation for every word independently in the pipeline?** That is exactly what it does; the labels come from averaged window logits.
24. **What would improve commas?** More comma data, balanced sampling, or a dedicated comma model.

25. **How is punctuation evaluated?** 1,500 test windows, token accuracy, boundary P/R/F1, per-class F1.

29.3 Ordering

26. **Why pairwise?** Easy supervision, small model, and it generalizes to any document length.
27. **What is the input format?** `[CLS] sentence A [SEP] sentence B [SEP]`.
28. **What is the label?** 1 if A precedes B, 0 otherwise.
29. **Why `ORDER_MAX_LENGTH=192`?** Two 90-token sentences plus separators fit.
30. **What is Borda count?** Each pair vote adds to the winner's score; sort by score.
31. **Why Borda over topological sort?** Preferences can be cyclic; Borda tolerates cycles.
32. **What are the three decoders?** Borda, clusters (greedy), beam (incremental).
33. **Which won?** Borda: tau 0.2007 vs clusters 0.1223 and beam 0.0921.
34. **Why is tau low?** Pairwise noise, cycles, and error propagation.
35. **What is Kendall's tau?** Rank correlation between predicted and true order.
36. **Why a TF-IDF ablation?** It isolates how much the neural model adds over lexical overlap.
37. **What did the ablation show?** TF-IDF tau 0.0631, much lower than the model.
38. **How is ordering evaluated?** Pair accuracy/macro F1 on 3,000 pairs, plus tau on 40 shuffled articles.
39. **Why 40 articles?** It is enough for a stable tau estimate and cheap to run.
40. **What is the complexity?** $O(n^2)$ pairs per article; for 100 sentences, ~4,950 pairs.

29.4 Translation

41. **Why IndicTrans2?** Indic-specialized, strong Hindi→English, 200M is feasible.
42. **What is the language code pair?** `hin_Deva` → `eng_Latn`.
43. **Why the toolkit?** It handles Indic tokenization/spacing and the model's processor.
44. **Why not pass `src_lang` / `tgt_lang` to the tokenizer?** This model does not take them; passing them breaks.
45. **What is `_import_indic_processor`?** A compatibility shim for transformers 4 vs 5 import paths.
46. **Why `return_attention_mask`?** IndicTrans2's forward needs it explicitly.
47. **Why `clean_up_tokenization_spaces=False`?** It preserves Indic spacing.
48. **Why a local mirror?** The official checkpoint is gated; a mirror was available.
49. **What are BLEU and chrF?** n-gram precision/recall and character n-gram F-score.
50. **Why evaluate on IITB?** It is a standard Hindi-English benchmark with references.
51. **What is the 256-token limit?** The model's max positions; long sentences are truncated.

29.5 Engineering

52. **Why whitespace tokenization?** `\w+` drops Devanagari matras.
53. **Why pin transformers 4.44.2?** transformers 5 removes the ONNX/tokenizer base path IndicTrans2 needs.
54. **Why pin peft 0.13.2?** Newer peft imports `torch.distributed.tensor`, absent on macOS torch.
55. **Why pin datasets 3.6.0?** datasets 4/5 require huggingface-hub ≥ 1.31, conflicting with the stack.
56. **Why eager attention on MPS?** SDPA with dropout is broken on MPS.

57. **Why fp32 on MPS?** fp16 is CUDA-only in this stack.
58. **How is memory capped?** `torch.cuda.set_per_process_memory_fraction(1.5/total)`.
59. **Why stage the models?** Loading all three exceeds 1.5 GB.
60. **Why no fallback model?** Silent fallbacks hide broken setups and hurt reproducibility.
61. **Why no LLM?** Assignment constraint; also cost, latency, determinism.
62. **How is the pipeline aligned?** Same sentence boundaries and `GROUP_SIZE` for Hindi and English.
63. **Why `GROUP_SIZE=5`?** Groups are readable and match the expected output format.
64. **What happens to `NULL`?** Removed, counted, reported.
65. **How do you reproduce?** Pinned env, documented commands, saved reports, executed demo.

Appendices

Appendix A. Command reference

```
# Environment
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-colab.txt
pip install IndicTransToolkit==1.1.1

# Data
python -m src.data_prep --num-articles 3000 --max-train-pairs 40000

# Training
python -m src.punctuation.train --epochs 2 --batch-size 16 \
    --learning-rate 3e-4 --max-train-samples 4000
python -m src.ordering.train --epochs 1 --batch-size 16 \
    --learning-rate 2e-4 --max-train-samples 4000

# Evaluation
python -m src.evaluate \
    --translation-model models/translation/indictrans2-indic-en-200M \
    --max-translation-examples 100

# Pipeline
python -m src.pipeline \
    --translation-model models/translation/indictrans2-indic-en-200M \
    --translation-batch-size 8 --strategy borda

# Focused evaluations
python -m src.evaluate --skip-ordering --skip-translation --report /tmp/p.json
python -m src.evaluate --skip-punctuation --skip-translation --report /tmp/o.json
python -m src.evaluate --skip-punctuation --skip-ordering --report /tmp/t.json

# Smoke test
python -m src.pipeline --limit 1 --output-dir /tmp/smoke

# Build the guide
python scripts/build_study_guide.py
```

Appendix B. Hyperparameters

Parameter	Value
`BASE_MODEL`	`bert-base-multilingual-cased`
`TRANSLATION_MODEL`	`ai4bharat/indictrans2-indic-en-dist-200M`
`SRC_LANG` / `TGT_LANG`	`hin_Deva` / `eng_Latn`

Parameter	Value
`MAX_SEQ_LENGTH`	128
`ORDER_MAX_LENGTH`	192
`WINDOW_WORDS` / `STRIDE_WORDS`	64 / 40
`MIN_WORDS` / `MAX_WORDS`	32 / 96
`MIN_SENT_WORDS`	5
`MAX_SENT_WORDS`	40
`GROUP_SIZE`	5
`SEED`	42
LoRA r / alpha / dropout	16 / 32 / 0.1
LoRA targets	`query`, `value`
Punctuation LR / batch / epochs	3e-4 / 16 / 2 (subset)
Ordering LR / batch / epochs	2e-4 / 16 / 1 (subset)
Memory cap	1.5 GB

Appendix C. Dataset statistics

Split	Articles	Notes
train	2,700	window/pair generation
val	150	model selection
test	150	evaluation

Dataset	Train	Val	Test
Punctuation windows	25,000	2,317	2,208
Ordering pairs	40,000	3,000	3,000
Ordering eval articles	—	—	40

Label	Train count	Share
none	1,653,107	90.9%
comma	59,324	3.3%
sentence_end	104,840	5.8%
question	236	~0.01%
exclamation	153	~0.01%
total words	1,817,660	100%

Appendix D. Report schemas

`reports/data_prep_summary.json`:

```
{
  "articles": {"train": 2700, "val": 150, "test": 150},
  "punctuation": {"train": 25000, "val": 2317, "test": 2208},
  "ordering": {"train": 40000, "val": 3000, "test": 3000,
               "eval_articles": 40},
  "label_counts": {"none": 1653107, "comma": 59324,
                   "sentence_end": 104840,
                   "question": 236, "exclamation": 153}
}
```

`models/*/training_summary.json`:

```
{
  "base_model": "bert-base-multilingual-cased",
  "trainable_parameters": 593669,
  "total_parameters": 177860362,
  "train_metrics": {},
  "eval_metrics": {"eval_loss": 0.4399, "eval_boundary_f1": 0.8246},
  "test_metrics": {"test_accuracy": 0.8688, "test_boundary_f1": 0.8633},
  "peak_memory_gb": 0.0
}
```

`reports/run_summary.json`:

```
{
  "strategy": "borda",
  "timings": {"punctuation_seconds": 2.31,
              "ordering_seconds": 130.53,
              "translation_seconds": 102.17},
  "files": {"sample-1": {"words": 747, "null_removed": 5,
                         "sentences": 52, "groups": 11}}
}
```

Appendix E. Dependency error table

Symptom	Cause	Fix
`ImportError: transformers.onnx`	transformers 5 removed it	pin 4.44.2
`ModuleNotFoundError: torch.distributed.tensor`	peft too new for macOS torch	pin peft 0.13.2
`huggingface-hub` conflict	datasets 4/5 requires hub ≥ 1.31	pin datasets 3.6.0

Symptom	Cause	Fix
SDPA dropout error on MPS	MPS SDPA bug	use eager attention
fp16 error on MPS	fp16 is CUDA-only here	use fp32
CUDA OOM on T4	three models or large batches	stage models, cap memory, smaller batch
Gated model 401	no license acceptance	accept license, log in, or use mirror
Toolkit build failure	Rust/compiler missing on Colab	use wheel or source build per setup script
Tokenizer `src_lang` error	IndicTrans2 does not accept it	call tokenizer without `src_lang` / `tgt_lang`
Text extraction shows broken matras	PDF font/ToUnicode issue	visual shaping is correct; avoid text search

Appendix F. References and further study

- Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (2019).
- Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models* (2021).
- Gala et al., *IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages* (2023).
- Vaswani et al., *Attention Is All You Need* (2017).
- Koehn, *Neural Machine Translation* (2020).
- Jurafsky and Martin, *Speech and Language Processing* (3rd ed. draft).
- Hugging Face `transformers`, `peft`, `datasets` documentation.
- AI4Bharat `IndicTrans2` and `IndicTransToolkit` repositories.
- `cfilt/iitb-english-hindi` dataset on Hugging Face.
- NLTK book, chapters on tokenization and stemming.

Appendix G. Study plan

Week 1 — foundations. Tokens, embeddings, attention, transformer blocks, BERT pretraining. Run the three lesson notebooks. Be able to draw a transformer block from memory.

Week 2 — finetuning and adapters. Classification heads, cross-entropy, class weights, LoRA math, training loops. Read `src/punctuation/train.py` and `src/ordering/train.py` line by line.

Week 3 — the data pipeline. Wikipedia parsing, sentence splitting, sliding windows, label alignment, balanced pairs, splits. Re-run `src.data_prep` and inspect the JSONL.

Week 4 — inference and decoding. Sliding-window prediction, Borda/clusters/beam, Kendall tau. Re-run `src.evaluate` and reproduce the reported numbers.

Week 5 — translation and orchestration. IndicTrans2, tokenizer quirks, staged loading, memory caps. Run the pipeline end to end and inspect all four outputs per file.

Week 6 — communication. Read this guide's Parts 0.5 and IV, answer the viva bank aloud, and explain one design decision to someone else in two minutes.